SOFTWARE ARCHITECTURE • ENGINEERING PRACTICE

项目架构设计与开发理解文档

数据中心动环监控平台技术白皮书｜从真实代码理解系统边界、数据流、权限模型与演进路径

项目	dcim-iot-platform
代码基线	main · 5d26eeb
文档版本	技术白皮书版 1.0
生成日期	2026-07-12
阅读对象	开发者、维护者、架构复盘人员

Internal Engineering Documentation · Long-term Reference Edition

CHAPTER

目录

目录由当前增强版正文自动生成；页码在首轮排版后回填，并与 PDF 书签保持一致。

一、项目整体架构分析5

1. 项目定位 5
2. 当前系统架构 6
3. 架构图 8

二、模块划分与依赖分析13

1. 前端交互模块 13
2. 账号、认证与初始化模块 13
3. 组织、成员与申请模块 14
4. 权限与数据范围模块 14
5. 资源目录模块 15
6. 测点与指标模块 15
7. 旧遥测与旧告警兼容模块 16
8. 权威告警事件与通知模块 16
9. 仿真模块 17
10. 看板、任务与横切基础设施模块 17

三、核心业务流程与数据流分析19

1. 登录与请求认证流程 19
2. 公开注册、邮箱验证码与首次初始化流程 20
3. 组织创建申请与供应流程 21
4. 组织加入、审批与当前组织切换流程 22
5. 数据范围解析与资源访问流程 23
6. 机房、设备与导入流程 24
7. 测点模板、绑定与指标写入流程 25
8. 旧遥测与阈值告警流程 27
9. 权威告警状态流转与实时通知流程 28
10. 仿真场景运行流程 29
11. 看板与后台任务流程 30

四、模块详细设计规范31

1. 账号认证模块 31
2. 组织与成员模块 32
3. 权限与数据范围模块 33
4. 资源目录模块 34
5. 测点与指标模块 35
6. 旧遥测与规则告警模块 36
7. 权威告警与通知模块 36
8. WebSocket 模块 37
9. 仿真模块 38
10. 横切基础设施模块 39

五、异常处理机制分析41

1. 异常分类 41
2. 异常产生位置 41
3. 异常传播流程 42
4. 全局异常处理 42
5. 当前异常处理问题 42

六、关键设计决策分析44

1. 全局账号 + 组织成员关系 44
2. 双层权限：入口 RBAC + Service 数据范围 45
3. MySQL 单库 + Flyway 追加迁移 46
4. Redis 用于短期安全状态和缓存 47
5. 历史指标 + 最新快照双表 48

6. 两套遥测/告警模型并存 49
7. WebSocket 摘要 + REST 回源 50
8. 仿真复用正式指标写入 51

七、WebSocket 实时通知专项分析52

1. 使用场景 52
2. 生命周期 52
3. 消息流程 53
4. 数据关系 53
5. 当前限制 53

八、工程经验专题54

1. Spring Security 与组织数据权限 54
2. Redis 缓存与短期安全状态 55
3. WebSocket 长连接与用户级通知 56
4. MySQL 数据模型、索引与查询 57
5. 事务提交后副作用与异步任务 58

九、性能分析与系统演进59

1. 当前已实现的优化 59
2. 当前性能限制 60
3. 架构演进方向 60

十、技术债与设计问题分析62

P0：应在下一轮功能开发前处理 63
P1：影响一致性、可维护性或中期运行 64
P2：应在持续维护中收敛 65

十一、开发者快速理解总结66

1. 建议的代码阅读顺序 66
2. 新开发者必须建立的四个认知模型 66
3. Debug 快速决策路径 67
4. 功能扩展前的边界检查 67
5. 当前最合理的下一步 67

附录 A：关键代码目录索引68

附录 B：数据库迁移认知索引69

本文不替代 `README.md` 的部署与使用说明，也不重复 `AGENTS.md` 的 AI 协作和编码约束。本文关注当前代码为什么形成这样的结构、模块如何协作、数据如何流动、异常如何传播，以及后续维护时最容易误判的边界。

阅读结论先行

当前系统是一个**模块化单体方向的分层单体应用**：物理上只有一个 Spring Boot 后端进程、一个 Vue 前端应用和一套共享 MySQL 数据库；逻辑上已形成账号、组织、资源、测点、告警、通知、仿真等业务边界。它不是分布式系统，也尚未达到严格模块化单体，因为模块仍共享实体、Mapper、数据库和部分兼容链路。

系统当前最重要的认知点是：

1. 身份模型已经从“用户表上直接挂组织和角色”演进为“全局账号 + 组织成员关系”，请求组织由 `X-Organization-Id` 选择。
2. 遥测与告警同时存在两套链路：`device_data → alerts` 是旧兼容链，`metric_sample/metric_latest → alarm_event` 是新模型；新告警状态机尚无生产事件入口。
3. 权限不是只靠 URL 角色判断，而是“Spring Security 入口授权 + 组织上下文 + Service 数据范围校验”的组合。
4. WebSocket 不承载完整业务数据，只发送用户级摘要，前端收到后重新请求 REST，降低了实时消息与页面数据模型的耦合。
5. 仿真模块直接复用真实指标写入服务，因此能验证测点链路，但也要求生产环境安全开关、组织上下文传播和数据清理能力同时可靠。

30 分钟阅读路线

0-5 分钟：阅读“结论先行”和系统架构图，建立单体部署、逻辑模块和双告警链认知。

5-15 分钟：阅读模块依赖、权限范围、指标写入和告警通知流程，理解数据如何穿过系统。

15-23 分钟：阅读关键设计决策与工程经验专题，掌握方案背后的取舍。

23-30 分钟：阅读技术债优先级和 Debug 路径，形成后续维护行动清单。

CHAPTER

一、项目整体架构分析

本章要点

系统是单进程、单数据库的分层单体，同时具备清晰的逻辑业务模块。当前应先强化边界，不应把基础设施组件误判为分布式系统。

1. 项目定位

1.1 项目解决的问题

项目面向数据中心动环监控场景，核心目标是把机房、设备、测点、采集数据、告警处置和组织权限放进一个可运营的平台。当前代码已经覆盖：

- 全局账号注册、邮箱验证码、登录、首次平台管理员初始化；
- 组织创建申请、加入申请、成员审批和组织角色；
- 机房、设备类型、设备以及 Excel 导入导出；
- 测点字典、设备类型测点模板、设备测点绑定；
- 指标历史值与最新值；
- 旧版设备遥测、阈值规则、离线告警和邮件通知；
- 新版告警事件状态机、站内通知和用户级 WebSocket；
- 仿真场景、步骤、异步运行和生成数据追踪；
- 首页统计、审计、缓存、限流、定时任务和数据迁移。

核心业务场景不是单纯的设备 CRUD，而是：平台管理员治理组织，组织管理员管理本组织资源和成员，操作员处理被授权机房的业务，监控员只读查看；设备数据进入后形成最新状态、历史趋势和告警，告警再进入通知与处置流程。

1.2 当前规模

代码基线约包含：

范围	规模
后端主代码	282 个 Java 文件，约 16,531 行
后端测试	64 个 Java 文件，约 8,135 行
前端	97 个 Vue/JavaScript 文件，约 8,123 行
Controller	30 个
Service 实现	42 个
Mapper	29 个
Entity	29 个
DTO/VO	55 个
Flyway 迁移	V1 至 V10

这已经超出简单演示 CRUD 的规模，但仍属于单团队可维护的中型业务系统。当前阶段是“第一版产品能力已贯通，正在从基础项目向可运营平台演进”，主要矛盾已经从有没有功能转为：新旧模型收敛、权限语义一致、异步可靠性和数据生命周期闭环。

1.3 架构类型判断

结论：物理上是分层单体，逻辑上是模块化单体雏形。

判断依据：

- 后端全部编译进同一个 Spring Boot artifact，模块之间是进程内 Java 调用；
- 所有业务使用同一个 MySQL schema，没有服务级数据所有权；
- Redis、邮件和 WebSocket 是基础设施，不等同于微服务；
- 前端是单个 Vue SPA；
- 账号、组织、资源、指标、告警、仿真已形成相对独立的 Controller/Service/Entity 集合；
- 但模块间仍可直接引用对方 Service、Entity 和 Mapper，数据库外键与事务跨越多个业务域。

因此它不是分布式系统，也不应过早套用服务发现、消息总线和分布式事务。当前更合理的目标是先强化模块边界和事件接口，再根据真实扩展压力拆分。

2. 当前系统架构

2.1 前端层

前端采用 Vue 3、Vue Router、Pinia、Axios、Element Plus、ECharts 和 STOMP/SockJS。

职责：

- 页面编排、表单校验、表格和图表展示；

- 登录态、用户信息、当前组织和数据范围状态管理；
- 根据平台角色、组织角色和成员关系控制路由可见性；
- 给 REST 请求自动附加 `Authorization` 和 `X-Organization-Id`；
- 建立用户级 WebSocket 订阅，收到摘要后刷新 REST 数据。

前端不应承担最终授权。其权限控制主要改善交互，真正的数据边界仍由后端过滤器和 Service 校验完成。

关键入口：

- `frontend/src/router/index.js`
- `frontend/src/stores/auth.js`
- `frontend/src/stores/organization.js`
- `frontend/src/stores/scope.js`
- `frontend/src/utils/request.js`
- `frontend/src/api/realtime.js`

2.2 API 接口层

Controller 负责 HTTP 契约、参数校验、当前用户提取和统一结果封装。请求进入 Controller 前依次经过安全与横切组件：

1. `TraceIdFilter` 建立请求追踪标识并写入 MDC；
2. `RateLimitFilter` 使用 Redis 限流，Redis 异常时选择降级放行；
3. `XssFilter` 包装请求参数；
4. JWT 过滤器解析 Bearer Token，校验签名和黑名单；
5. `OrganizationContextFilter` 根据 `X-Organization-Id` 加载活动成员关系；
6. Spring Security 按 URL、HTTP 方法和角色做入口授权；
7. `@AuditLog` 切面记录成功或失败的敏感操作。

Controller 返回统一 `Result<T>`，业务异常最终由 `GlobalExceptionHandler` 转换。

2.3 业务逻辑层

Service 是系统真正的业务边界，承担：

- 组织角色和数据范围二次校验；
- 跨表一致性和事务；
- 状态机、唯一性、最后管理员保护等业务规则；
- DTO、Entity、VO 之间转换；
- 缓存、邮件、WebSocket、异步执行等基础设施协调；
- 兼容旧模型与新模型。

系统中多个 Controller 即使已有 `@PreAuthorize`，Service 仍重新检查组织、机房和设备归属。这是必要设计，因为后台任务、异步执行和内部 Service 调用可能绕过 HTTP 授权入口。

2.4 数据访问层

数据访问主要使用 MyBatis-Plus：

- `Mapper` 继承 `BaseMapper` 处理标准 CRUD；
- 复杂分页、聚合、锁和条件更新使用自定义 SQL 或 Wrapper；
- 分页插件限制最大页大小；
- Flyway V1-V10 管理 schema 演进；
- 部分关键流程使用 `FOR UPDATE`、CAS 条件更新和事务保证一致性。

MySQL 是最终事实源。Redis 主要是性能与短期状态设施，不承担核心业务永久存储。

2.5 基础设施层

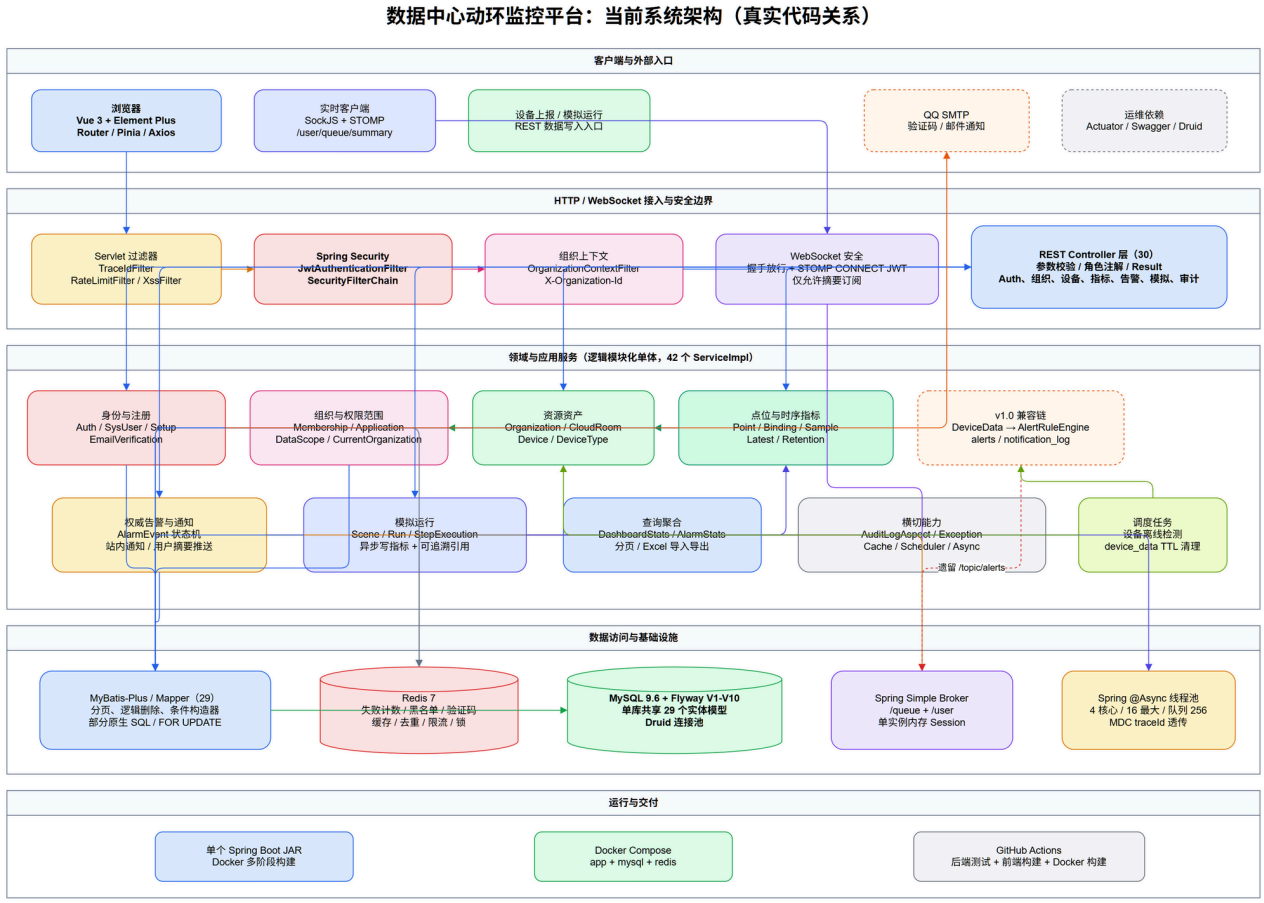
组件	当前用途	设计含义
MySQL	用户、组织、资源、指标、告警、通知、仿真、审计	单一事实源，支持本地事务
Redis	验证码、冷却时间、登录失败计数、Token 黑名单、限流、缓存、互斥锁	可降级能力和短期状态
Spring Mail	QQ 邮箱验证码及旧告警邮件	外部副作用，不能替代站内事实记录
STOMP/SockJS	用户级告警摘要实时推送	单机简单代理，无持久化
Async Executor	仿真运行异步执行	当前为进程内任务，不具备宕机恢复
Spring Retry	旧遥测写入死锁重试	缓解并发写冲突，不解决业务幂等
Flyway	数据库版本演进和修复	迁移必须追加，不回写历史环境

2.6 典型调用关系

```
Vue 页面
  ↓ Axios（JWT + 当前组织）
Filter / Spring Security
  ↓
Controller（参数和接口契约）
  ↓
Service（权限、业务规则、事务、状态机）
  ↓
Mapper / Cache / Mail / WebSocket / Async
  ↓
MySQL / Redis / QQ SMTP / 浏览器客户端
```

3. 架构图

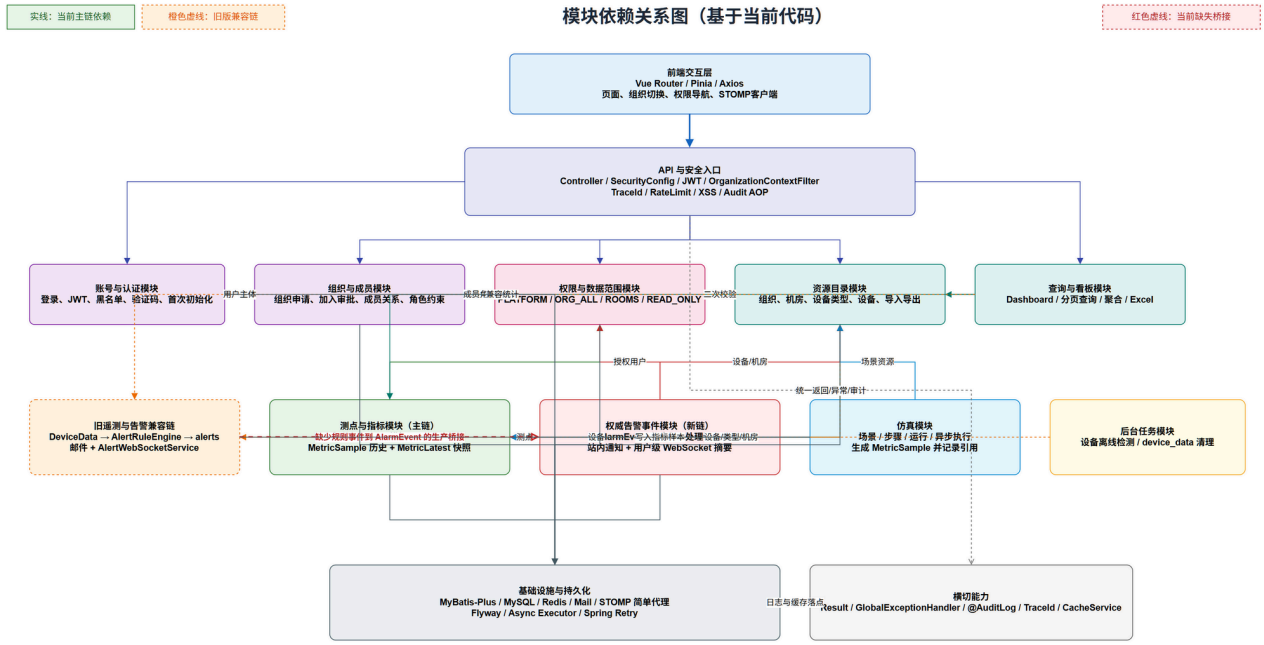
3.1 当前系统架构图



当前系统架构图

图中按真实运行边界展示 Vue SPA、Spring Security/API 层、业务模块、MySQL、Redis、QQ SMTP、STOMP 和异步任务。需要注意：WebSocket Broker 与异步执行器都在同一个后端进程内。

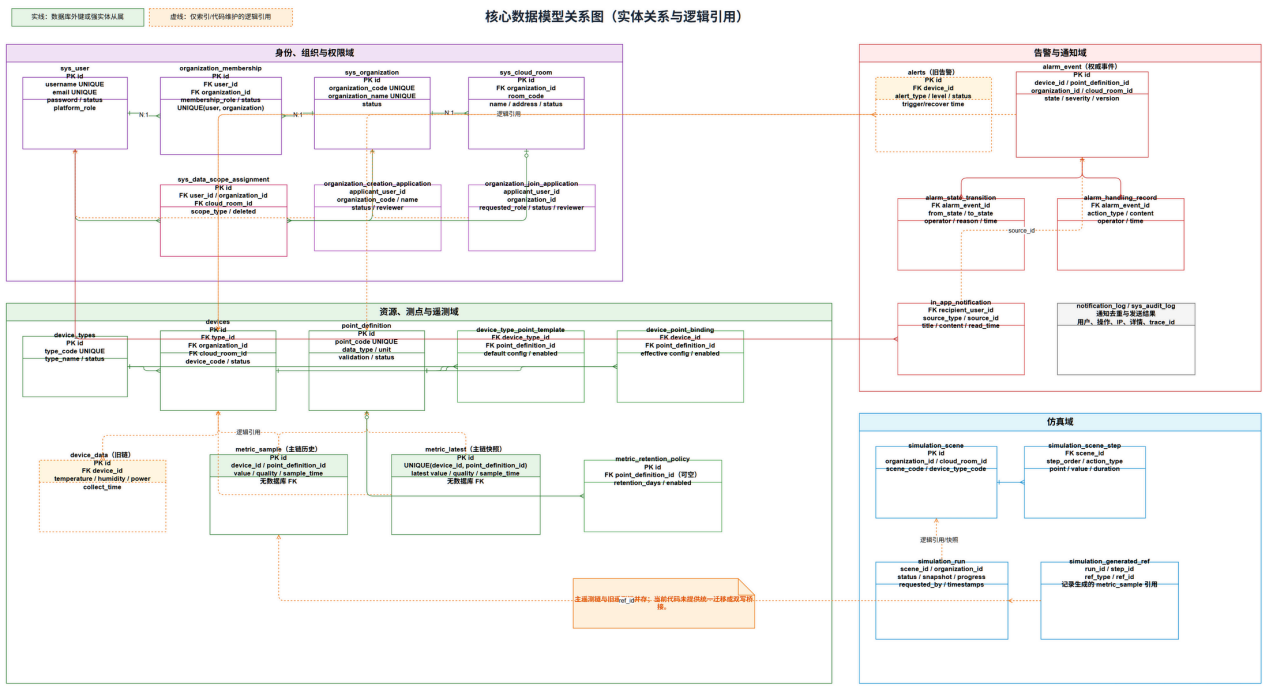
3.2 模块依赖关系图



模块依赖关系图

橙色虚线表示旧版兼容链，红色虚线表示代码模型存在但当前生产流程尚未接通的关系。

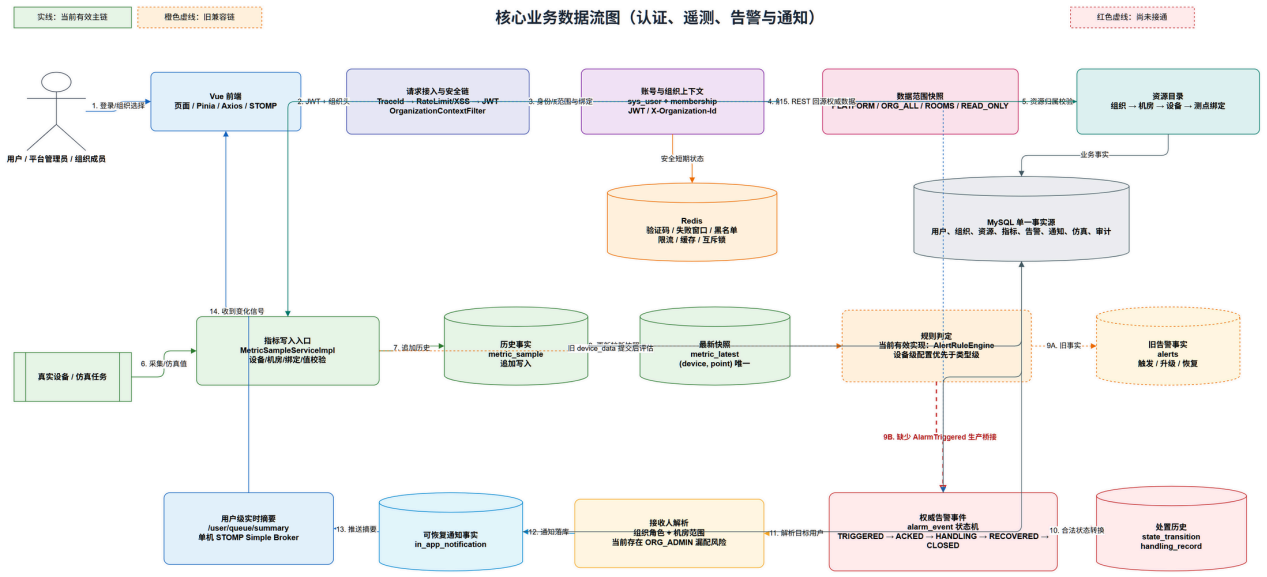
3.3 核心数据模型关系图



核心数据模型关系图

实体图特意区分数据库外键和仅由代码维护的逻辑引用。`metric_sample`、`metric_latest`、`alarm_event` 和部分仿真表为了写入或演进灵活性没有建立全部外键，因此一致性依赖 Service 层。

3.4 核心业务数据流图



核心业务数据流图

读图提示

图中绿色实线表示当前产品化指标主链，橙色虚线表示旧版兼容链，红色虚线表示规则结果到 `alarm_event` 的生产桥接尚未实现。实时摘要不是事实源，前端仍通过 REST 回源数据库。

CHAPTER

二、模块划分与依赖分析

本章要点

账号与组织提供治理基础，资源提供业务坐标，指标与告警形成数据闭环，仿真用于验证。权限范围是横跨模块的高耦合能力。

1. 前端交互模块

模块职责

负责把后端能力组织成可操作界面，维护登录态、当前组织、路由和实时刷新。它不负责最终授权，也不直接定义后端业务状态机。

核心能力

登录与注册页面、组织选择、资源管理、测点与指标页面、告警处置、仿真运行、平台管理、数据看板和通知入口。

依赖关系

依赖所有 REST Controller 的稳定接口、`Result<T>` 返回结构、JWT、组织请求头和 WebSocket 目的地。后端不会反向依赖前端，但 WebSocket 摘要结构与前端刷新策略形成接口契约。

边界与服务化

前端模块边界总体清晰，但路由角色判断、菜单显示和后端权限规则必须同步维护。它不涉及微服务拆分，未来可按业务域拆分前端 feature 目录或微前端，但当前规模没有必要。

2. 账号、认证与初始化模块

模块职责

负责全局账号、邮箱验证、密码登录、JWT 生命周期、平台注册开关和第一个平台管理员初始化。它不负责组织内角色，组织角色来自 `organization_membership`。

核心能力

- 邮箱或全局唯一用户名登录；
- 公开账号注册开关；
- 邮箱验证码 5 分钟有效、IP/邮箱 60 秒冷却及小时配额；
- Redis Lua 登录失败窗口；
- JWT 签发、单 Token 与用户级黑名单；

- Bootstrap Token + 邮箱验证码的一次性平台初始化。

依赖关系

依赖 `sys_user`、注册设置表、Redis、PasswordEncoder、JWT、邮件服务和审计。组织模块依赖它提供的用户主体；权限模块依赖认证结果中的用户 ID 和平台角色。

边界与服务化

职责相对独立，未来最适合优先形成独立身份服务。但当前组织上下文、SecurityContext 和用户表被广泛引用，拆分前需先定义认证声明、用户查询接口和角色同步机制。

3. 组织、成员与申请模块

模块职责

负责组织生命周期、成员关系、组织创建申请、加入申请、审批和组织管理员约束。它不保存平台身份凭证，也不负责具体机房授权明细。

核心能力

- 普通用户提交组织创建申请；
- 平台管理员审批并原子化创建组织与首个 `ORG_ADMIN`；
- 用户申请加入组织；
- 组织管理员审批 `OPERATOR` / `VIEWER` 等角色；
- 组织管理员不得同时属于其他组织；
- 删除或降级最后一个活动组织管理员时加锁保护。

依赖关系

依赖用户、组织、成员和申请 Mapper；审批后调用组织供应服务。数据范围模块依赖成员角色，资源模块依赖组织实体，前端组织 Store 依赖成员列表。

边界与服务化

业务独立性较强，但与用户、机房、权限均有共享数据。未来可形成租户/组织服务；拆分成本主要在跨服务成员校验和组织删除的一致性。

4. 权限与数据范围模块

模块职责

把“当前用户以什么组织角色访问哪个组织、机房和设备”转换成可执行的数据范围。它不负责认证凭证，也不负责前端菜单。

核心能力

- `PLATFORM`、`ORG_ALL`、`ROOMS`、`READ_ONLY` 四类范围；

- 从请求头和活动成员关系建立组织上下文；
- 将机房授权传播到设备、指标、告警和仿真；
- 提供 Service 级资源访问断言。

依赖关系

依赖 `organization_membership`、`sys_data_scope_assignment`、组织上下文和资源归属。几乎所有组织业务模块都会调用它，是当前耦合度最高的共享业务模块之一。

边界与服务化

作为独立微服务并不理想，因为每次业务操作都需要低延迟授权。更适合保持为统一授权库或策略引擎，并通过稳定的 `ScopeSnapshot`/策略接口被其他模块使用。当前 `DataScopeContext` 的若干帮助方法与范围语义不完全一致，是需要优先收敛的边界问题。

5. 资源目录模块

模块职责

管理组织下的机房、设备类型和设备资产，提供后续测点、告警、仿真的基础资源坐标。它不负责高频指标写入，也不直接处理告警状态。

核心能力

机房 CRUD、设备类型 CRUD、设备 CRUD、设备在线信息、设备导入导出、类型删除关联校验、组织和机房归属校验。

依赖关系

依赖组织和数据范围；测点、遥测、告警、仿真、看板都依赖资源目录。设备删除和类型删除会受到下游关联数据约束。

边界与服务化

是典型资产目录，可在未来拆成设备资产服务。但当前设备实体被指标、告警和仿真直接引用，拆分前需要建立设备快照或只读查询契约，避免每次高频采集都同步远程查询。

6. 测点与指标模块

模块职责

将不同设备的监测项标准化为测点定义，把类型默认测点复制为设备有效绑定，并存储历史指标和最新快照。它不负责告警状态流转。

核心能力

- 全局测点字典；
- 设备类型测点模板；

- 设备实例测点绑定；
- 指标值类型和范围校验；
- `metric_sample` 历史写入；
- `metric_latest` 最新值更新；
- 保留策略配置。

依赖关系

依赖资源目录和数据范围。仿真复用指标写入；看板读取最新值；新告警模型引用设备和测点，但当前缺少规则引擎到 `alarm_event` 的创建桥接。

边界与服务化

高频写入、数据增长和查询模式与管理类业务不同，未来适合作为独立遥测服务，甚至将历史数据迁移到时序数据库。拆分前必须解决设备/测点元数据缓存、写入幂等、最新值原子更新和告警事件投递。

7. 旧遥测与旧告警兼容模块

模块职责

维持原始 `device_data` 数据模型、阈值规则、`alerts` 表、离线检测、邮件和旧 WebSocket 推送。它主要承担历史兼容，不应继续扩展为新告警主模型。

核心能力

旧遥测写入、设备最后在线时间更新、规则匹配、告警生成/升级/恢复、通知去重、邮件发送、离线任务和旧数据清理。

依赖关系

依赖设备、告警配置缓存、Redis、邮件、审计和定时任务。看板仍有部分兼容统计依赖它。它与新版 `alarm_event` 当前没有生产桥接。

边界与服务化

不建议独立服务化，应该先定义退役计划：将规则结果转为统一告警领域事件，迁移历史读取，再逐步停止 `device_data` / `alerts` 写入。

8. 权威告警事件与通知模块

模块职责

负责告警事件生命周期、状态转换历史、处理记录、站内通知和面向用户的实时摘要。它不负责采集值判断是否触发告警，事件应由规则模块创建。

核心能力

`TRIGGERED → ACKED → HANDLING → RECOVERED → CLOSED` 状态机，受控快捷转换，乐观条件更新，操作记录，通知落库，按接收人推送 `/user/queue/summary`。

依赖关系

依赖设备、测点、组织/机房、数据范围、接收人解析、站内通知和 WebSocket。前端告警页和通知页依赖它。当前规则引擎尚未调用其创建能力。

边界与服务化

告警处理具有独立状态和扩展压力，是未来服务化候选。但必须先完成事件生产入口、接收人规则和可靠消息机制，否则拆分只会放大现有断链。

9. 仿真模块

模块职责

用可配置场景和步骤生成受控测试指标，验证设备、测点、指标、页面和后续告警链路。它不应成为生产采集入口。

核心能力

场景和有序步骤、运行快照、事务提交后异步启动、状态 CAS、取消轮询、MDC 与组织上下文传播、指标样本写入、生成引用追踪、生产环境禁用保护。

依赖关系

依赖资源目录、测点绑定、指标写入、数据范围、事务事件、异步执行器和 MySQL。当前如果告警桥接缺失，仿真只能验证指标和页面，不能完整验证新版告警闭环。

边界与服务化

适合作为开发/测试辅助模块，不需要优先服务化。若未来用于大规模压测，应成为外部测试工具，通过公开采集接口写入，而不是与生产应用共享进程线程池。

10. 看板、任务与横切基础设施模块

模块职责

看板负责读模型聚合；定时任务负责离线检测和清理；横切基础设施负责缓存、审计、异常、追踪、限流、邮件和 WebSocket。它们不应直接重新定义业务状态。

核心能力

30 秒看板缓存和互斥锁、分页和聚合、设备离线扫描、旧遥测清理、统一异常转换、审计日志、缓存降级、请求追踪。

依赖关系

这些组件横跨所有业务模块。高复用意味着任何语义变化都会有较大影响，尤其是 `Result<T>`、异常类型、缓存键、审计切面和组织上下文。

边界与服务化

看板可在查询压力上升后独立为读模型；定时任务可迁移到独立 Worker。异常、审计和权限更适合以库和统一规范存在，而不是单独远程服务。

CHAPTER

三、核心业务流程与数据流分析

本章要点

理解调用链时先确认当前请求属于账号组织流程、产品化指标链、旧告警兼容链还是新版告警状态链。链路判断错误会直接导致排障方向错误。

1. 登录与请求认证流程

1.1 业务目标

确认全局账号身份，限制暴力尝试，签发可撤销 JWT，并为后续组织上下文选择提供用户主体。

1.2 参与模块

`AuthController`、`AuthServiceImpl`、`SysUserMapper`、`CacheService`、JWT 组件、Spring Security、Redis、`AuditLogAspect`。

1.3 完整调用链

```
用户输入邮箱或全局唯一用户名、密码
  ↓
POST /api/auth/login
  ↓
AuthServiceImpl.login
  ├─ 按 email 或 username 查询 sys_user
  ├─ 检查账号状态、锁定窗口和密码
  ├─ Redis Lua 更新失败计数或清除失败状态
  └─ 签发包含 userId/platformRole/jti 的 JWT
  ↓
Result<LoginResponseDTO>
  ↓
前端保存 Token，后续请求附加 Authorization
```

JWT 不把组织角色固化进 Token。原因是同一用户可属于多个组织，组织角色取决于本次请求选择的组织，必须在请求时读取活动成员关系。

1.4 数据流

登录标识从 DTO 进入 Service，标准化后查询 `sys_user`。密码只参与 `PasswordEncoder.matches`，不进入返回值。失败计数和短时锁存在 Redis；长期账号状态存在 MySQL。JWT 由服务端签名，前端保存并随请求发送。登出时通过 jti 黑名单撤销单 Token，用户级黑名单时间戳可撤销该用户此前签发的全部 Token。

1.5 异常路径

用户不存在、密码错误、账号禁用和超出失败阈值均在 Service 抛出业务异常；Redis 计数是安全敏感操作，失败不能完全静默。JWT 解析失败在过滤器阶段转为未认证响应，不进入 Controller。审计切面记录登录成功或失败，但不得记录明文密码。

1.6 关键代码位置

- `controller/AuthController.java`
- `service/impl/AuthServiceImpl.java`：`login`
- `security/JwtAuthenticationFilter.java`
- `common/util/JwtUtil.java`
- `service/impl/CacheServiceImpl.java`

2. 公开注册、邮箱验证码与首次初始化流程

2.1 业务目标

允许普通用户创建全局账号，同时保证平台管理员只在部署初始化阶段一次性产生，避免开放接口创建平台特权账号。

2.2 参与模块

`AuthController`、`PlatformSetupController`、`PublicRegistrationServiceImpl`、`PlatformSetupServiceImpl`、`EmailVerificationServiceImpl`、注册设置 Service、邮件服务、Redis、`sys_user`、平台注册设置表。

2.3 完整调用链

公开注册：

```
用户请求验证码
  ↓
检查平台注册开关、IP/邮箱冷却和小时配额
  ↓
生成验证码，仅缓存 HMAC 摘要，TTL 5 分钟
  ↓ QQ SMTP
用户提交邮箱、验证码、用户名和密码
  ↓
校验验证码并原子消费
  ↓
检查 email/username 全局唯一
  ↓
创建无平台角色、无组织绑定的全局账号
```

首次初始化：

```
部署者配置 Bootstrap Token
  ↓
GET 初始化状态确认尚无 PLATFORM_ADMIN
  ↓
提交 Bootstrap Token + 邮箱验证码 + 账号信息
  ↓
PlatformSetupServiceImpl 在事务中锁定初始化状态
  ↓
再次确认不存在平台管理员
  ↓
创建第一个 PLATFORM_ADMIN
```

2.4 数据流

验证码明文只在生成时进入邮件，不进入数据库；Redis 保存带用途和邮箱绑定的摘要、冷却键与配额计数。注册完成后验证码被消费，用户主体进入 `sys_user`。Bootstrap Token 来自部署环境变量，不是普通用户输入的业务邀请码。

2.5 异常路径

平台注册关闭、邮箱格式错误、验证码过期/不匹配、冷却未结束、配额耗尽、用户名或邮箱重复会在业务层拒绝。初始化还可能因 Bootstrap Token 错误或平台管理员已存在而失败。并发初始化通过数据库锁和事务内二次检查收敛到一个成功者。

2.6 关键代码位置

- `service/impl/EmailVerificationServiceImpl.java`
- `service/impl/PublicRegistrationServiceImpl.java`
- `service/impl/PlatformSetupServiceImpl.java`：`initialize`
- `service/impl/PlatformRegistrationSettingsServiceImpl.java`
- `controller/PlatformSetupController.java`
- `resources/db/migration/V7__registration_identity_and_settings.sql`

3. 组织创建申请与供应流程

3.1 业务目标

把账号注册与付费/审核后的组织开通解耦。普通用户先拥有全局账号，再申请创建组织；平台管理员批准后才产生组织和首个组织管理员成员关系。

3.2 参与模块

`OrganizationCreationApplicationController` 、 `OrganizationCreationApplicationServiceImpl` 、 `OrganizationProvisioningServiceImpl`、组织/成员/申请 Mapper、平台权限、MySQL。

3.3 完整调用链

```
普通用户提交组织代码、名称和申请说明
  ↓
创建 PENDING 的 organization_creation_application
  ↓
平台管理员查询待审核申请
  ↓
approve
  ↓ 事务
锁定申请并检查状态
  ├─ 校验组织代码和名称唯一
  ├─ 创建 sys_organization
  ├─ 创建申请人为 ORG_ADMIN 的 ACTIVE membership
  └─ 更新申请为 APPROVED，记录 reviewer
  ↓
前端重新加载用户组织列表
```

3.4 数据流

申请表保存用户意图和审批轨迹；组织正式数据只在批准时写入。供应服务将“创建组织”和“授予首个管理员”放在同一事务，避免出现无管理员组织。用户表本身不再写入 `organization_id` 作为权威关系。

3.5 异常路径

重复待审申请、组织代码/名称冲突、申请已处理、申请人已是其他组织管理员、审批人无平台权限均会失败。事务保证组织或成员任一步失败时整体回滚。

3.6 关键代码位置

- `controller/OrganizationCreationApplicationController.java`
- `service/impl/OrganizationCreationApplicationServiceImpl.java`：`submit`、`approve`
- `service/impl/OrganizationProvisioningServiceImpl.java`
- `entity/OrganizationCreationApplication.java`
- `resources/db/migration/V8__organization_membership_and_applications.sql`

4. 组织加入、审批与当前组织切换流程

4.1 业务目标

允许全局账号申请加入已有组织，由该组织管理员审批角色，并让一个账号在前端切换其活动组织上下文。

4.2 参与模块

组织加入 Controller/Service、`OrganizationMembershipServiceImpl`、`OrganizationContextFilter`、前端 organization Store、`organization_membership`、`X-Organization-Id`。

4.3 完整调用链

```
用户浏览可申请组织并提交加入申请
  ↓
创建 PENDING join application
  ↓
目标组织 ORG_ADMIN 审批并指定允许角色
  ↓ 事务
校验审批人、申请状态、角色和跨组织管理员约束
  ↓
创建或激活 organization_membership
  ↓
前端选择组织并持久化 currentOrganizationId
  ↓
Axios 给后续请求附加 X-Organization-Id
  ↓
OrganizationContextFilter 加载该用户 ACTIVE membership
  ↓
当前请求获得 organizationId + membershipRole
```

4.4 数据流

组织选择只保存在前端状态和每次请求头中，不修改用户的永久“当前组织”。后端始终按用户 ID、组织 ID 和 ACTIVE 状态重新验证成员关系。这样同一用户可作为普通成员加入多个组织，但一旦成为任一组织 `ORG_ADMIN`，业务规则禁止其以任何角色加入其他组织。

4.5 异常路径

未加入组织却伪造请求头时，过滤器不会建立有效组织上下文；后续授权或 Service 校验失败。重复申请、无权审批、非法角色、跨组织管理员冲突和最后一个管理员退出都会被拒绝。最后管理员保护需要事务锁，避免并发操作同时通过计数检查。

4.6 关键代码位置

- `controller/OrganizationJoinApplicationController.java`
- `service/impl/OrganizationJoinApplicationServiceImpl.java`
- `service/impl/OrganizationMembershipServiceImpl.java`
- `security/OrganizationContextFilter.java`
- `frontend/src/stores/organization.js`
- `frontend/src/utils/request.js`

5. 数据范围解析与资源访问流程

5.1 业务目标

在同一组织内进一步限制操作员可访问的机房，同时让组织管理员拥有组织全量权限、监控员保持只读。

5.2 参与模块

`OrganizationContextFilter` 、 `DataScopeServiceImpl` 、 `UserDataScopeServiceImpl` 、 `sys_data_scope_assignment`、CloudRoom/Device/Metric/Alarm/Simulation Service。

5.3 完整调用链

```
已认证请求 + X-Organization-Id
  ↓
成员关系解析：PLATFORM_ADMIN 或 ORG_ADMIN/OPERATOR/VIEWER
  ↓
DataScopeServiceImpl.resolveCurrentScope
  ├─ 平台管理员无组织头：PLATFORM
  ├─ ORG_ADMIN：ORG_ALL
  ├─ OPERATOR 有机房授权：ROOMS
  ├─ OPERATOR 无授权：READ_ONLY
  └─ VIEWER：READ_ONLY
  ↓
业务 Service 检查 organizationId/cloudRoomId/deviceId
  ↓
允许查询/写入，或抛出 DataScopeViolationException
```

5.4 数据流

数据范围是一个请求期快照，来源是 SecurityContext、组织成员关系和机房授权表。机房范围进一步通过设备的 `cloud_room_id` 传播到指标、告警和仿真。Controller 的角色注解解决“能否调用此类接口”，Service 解决“能否操作这条数据”。

5.5 异常路径

组织头缺失、成员无效、组织不匹配、机房未授权、设备不在授权机房均会失败。当前实现存在一个重要风险：`ORG_ALL` 的 `cloudRoomIds` 通常为空，而部分代码直接调用 `hasCloudRoomAccess`，可能把“全组织访问”误判为“无机房访问”，详见技术债章节。

5.6 关键代码位置

- `security/OrganizationContextFilter.java`
- `service/impl/DataScopeServiceImpl.java`
- `service/impl/UserDataScopeServiceImpl.java`
- `security/CurrentOrganizationContext.java`
- `dto/DataScopeContext.java`

6. 机房、设备与导入流程

6.1 业务目标

建立可被采集、指标、告警和仿真引用的资源目录，并保证设备只能归属于当前组织内的机房和有效设备类型。

6.2 参与模块

`CloudRoomController/Service`、`DeviceController/Service`、`DeviceTypeServiceImpl`、Excel 导入组件、组织与数据范围服务、对应 Mapper 和 MySQL。

6.3 完整调用链

单条创建设备：

```
组织用户提交设备信息
  ↓
DeviceController 参数分组校验
  ↓
DeviceServiceImpl.save
  ├─ 校验组织上下文与写权限
  ├─ 校验机房属于组织且可访问
  ├─ 校验设备类型存在、编码受支持
  └─ 检查设备编码唯一并写入 devices
```

Excel 导入：解析每行 `organizationCode` 和 `cloudRoomCode`，逐行解析实体、校验数据范围并汇总错误，不能因为前端传入组织字段而跳过归属验证。

6.4 数据流

资源输入从 DTO/Excel 行转换为 Entity，组织和机房标识会解析为数据库 ID。设备记录成为指标、告警和仿真的稳定引用。设备类型删除前查询关联设备，存在引用时主动拒绝，避免产生悬空设备。

6.5 异常路径

组织或机房不存在、跨组织引用、数据范围不足、设备类型不存在、编码重复、导入字段格式错误都会产生业务或校验异常。删除设备类型时若仍有设备，`DeviceTypeServiceImpl.deleteById` 抛出 `IllegalStateException`，由全局异常处理转换；该异常类型比专用业务异常语义弱。

6.6 关键代码位置

- `controller/CloudRoomController.java`
- `service/impl/CloudRoomServiceImpl.java`
- `controller/DeviceController.java`
- `service/impl/DeviceServiceImpl.java`：`save`
- `service/impl/DeviceTypeServiceImpl.java`：`deleteById`
- `service/impl/DeviceServiceImpl.java`：`importFromExcelScoped`

7. 测点模板、绑定与指标写入流程

7.1 业务目标

用统一测点定义描述不同设备的数据项，通过类型模板降低配置成本，同时保留设备实例级覆盖，并维护历史值和最新值两个读写模型。

7.2 参与模块

PointDefinition 、 DeviceTypePointTemplate 、 DevicePointBinding 、 MetricSample 、 MetricLatest Controller/Service/Mapper，设备和数据范围服务，MySQL。

7.3 完整调用链

```
平台维护全局 PointDefinition
  ↓
为 DeviceType 配置测点模板
  ↓
设备创建/配置时形成 DevicePointBinding
  ↓
采集或仿真提交 MetricSampleDTO
  ↓
MetricSampleServiceImpl
  ├─ 校验设备和机房访问
  ├─ 校验绑定存在且启用
  ├─ 按测点 dataType/range 校验值
  ├─ 插入 metric_sample
  └─ 调用 MetricLatestService 更新较新的快照
  ↓
趋势查询读取 sample，状态页读取 latest
```

7.4 数据流

测点定义保存语义和校验规则，模板是类型默认值，绑定是设备真正生效的配置。指标样本不可变地追加到历史表；最新表以 `(device_id, point_definition_id)` 唯一定位，并只接受时间更晚的值。保留策略保存期限配置，但当前没有对应 `metric_sample` 清理任务。

7.5 异常路径

设备或绑定不存在、绑定禁用、值类型错误、数值越界、机房无权访问时拒绝写入。历史写成功但最新值更新失败时，事务边界决定是否整体回滚。当前最新值采用读后写，极端并发下仍有竞争风险。

7.6 关键代码位置

- `service/impl/PointDefinitionServiceImpl.java`
- `service/impl/DeviceTypePointTemplateServiceImpl.java`
- `service/impl/DevicePointBindingServiceImpl.java`
- `controller/MetricSampleController.java`
- `service/impl/MetricSampleServiceImpl.java`
- `service/impl/MetricLatestServiceImpl.java`
- `resources/db/migration/V3__device_point_metric_model.sql`

8. 旧遥测与阈值告警流程

8.1 业务目标

兼容基础项目的固定字段遥测模型，在写入温度、湿度、电力等数据后执行阈值规则，形成旧版告警并发送通知。

8.2 参与模块

`DeviceDataServiceImpl` 、 `AlertRuleEngine` 、 告 警 配 置 Service/Cache 、 `alerts` 、 `NotificationDedupServiceImpl` 、邮件服务、 `AlertWebSocketService` 、设备离线任务。

8.3 完整调用链

```
设备固定字段数据写入
  ↓ READ_COMMITTED + 死锁重试
更新 devices.last_online_time
  ↓
插入 device_data
  ↓ 事务提交后
AlertRuleEngine.evaluate
  ├─ 读取设备级配置，缺失则回退类型级配置
  ├─ 比较阈值并创建/升级/恢复 alerts
  └─ 以 alertId 做通知去重
       ├─ QQ 邮件
       └─ 旧 /topic/alerts 推送
```

8.4 数据流

`device_data` 保存固定字段快照，规则配置通过 cache-aside 读取。告警结果写入 `alerts` ，通知日志用于去重和结果记录。设备离线定时任务也直接生成同一旧告警模型。

8.5 异常路径

数据库死锁触发 Spring Retry；规则评估安排在事务提交后，避免读取未提交数据。通知失败不应回滚告警事实，但要写通知结果。当前旧 WebSocket 目的地与新 Broker 配置不一致，且前端只订阅新用户队列，因此旧实时推送很可能不再生效。

8.6 关键代码位置

- `service/impl/DeviceDataServiceImpl.java`
- `service/impl/AlertRuleEngine.java`
- `service/impl/NotificationDedupServiceImpl.java`
- `service/impl/EmailNotificationServiceImpl.java`
- `service/impl/AlertWebSocketService.java`
- `task/DeviceMonitorTask.java`

9. 权威告警状态流转与实时通知流程

9.1 业务目标

让一个告警事件具有可审计的处置生命周期，并把变化只推送给有权接收的用户。

9.2 参与模块

`AlarmEventController`、`AlarmEventServiceImpl`、状态转换 Mapper、处理记录 Mapper、`AlarmRecipientResolverImpl`、`InAppNotificationServiceImpl`、`UserScopedWebSocketPublisher`、STOMP Broker、前端 WebSocket 客户端。

9.3 完整调用链

```
用户执行确认/处理/恢复/关闭
  ↓
AlarmEventController
  ↓
AlarmEventServiceImpl.transition
  ├─ 查询事件并校验组织/机房范围
  ├─ 校验状态转换是否合法
  ├─ 条件更新当前状态
  ├─ 插入 alarm_state_transition
  └─ 插入 alarm_handling_record
  ↓ 事务提交后
解析有权接收用户
  ├─ 写 in_app_notification
  └─ convertAndSendToUser(userId, /queue/summary, WebSocketSummary)
  ↓
前端收到摘要，重新请求 REST 获取权威详情
```

9.4 数据流

`alarm_event` 是当前状态，transition 和 handling 是不可变历史，站内通知是用户级投递记录，WebSocket 摘要只是变化信号。完整告警内容始终从数据库经 REST 查询，断线后仍可恢复。

9.5 异常路径

非法状态转换、并发状态已变化、数据范围不足、事件不存在会在事务内失败，不写历史。通知或 WebSocket 失败不应破坏已提交状态。当前更关键的异常不是运行报错，而是**没有生产代码创建 `alarm_event`**；状态机完整但缺少规则事件入口。

9.6 关键代码位置

- `controller/AlarmEventController.java`
- `service/impl/AlarmEventServiceImpl.java`
- `service/impl/AlarmRecipientResolverImpl.java`
- `service/impl/InAppNotificationServiceImpl.java`
- `security/UserScopedWebSocketPublisher.java`

- `config/WebSocketConfig.java`

10. 仿真场景运行流程

10.1 业务目标

在不接真实硬件时，以可重复场景生成指标样本，验证资源、测点、指标查询和前端展示。

10.2 参与模块

SimulationScene/Run Controller 与 Service 、`SimulationRunExecutionServiceImpl` 、`SimulationStepExecutionServiceImpl`、异步执行器、事务同步、MetricSample Service、生成引用 Mapper。

10.3 完整调用链

```
用户创建场景及有序步骤
  ↓
POST 启动运行
  ↓
SimulationRunServiceImpl.startRun
  ├─ 检查 simulation.write-enabled 和非生产保护
  ├─ 检查场景、设备、机房、测点范围
  ├─ 保存 PENDING run 与场景快照
  └─ 注册事务提交后异步执行
  ↓
SimulationRunExecutionServiceImpl
  ├─ 传播 MDC 和组织上下文
  ├─ CAS PENDING → RUNNING
  ├─ 逐步骤执行并每约 200ms 检查取消
  ├─ 复用 MetricSampleServiceImpl 写指标
  └─ 记录 simulation_generated_ref
  ↓
更新 SUCCEEDED / FAILED / CANCELLED
```

10.4 数据流

运行记录保存启动时场景快照，避免后续编辑场景改变历史运行含义。每个步骤生成的指标样本进入正式 `metric_sample` / `metric_latest`，并通过 generated ref 反向追踪。异步线程手动恢复组织上下文，避免权限检查失去请求信息。

10.5 异常路径

安全开关关闭、生产环境、资源无权、场景无步骤、绑定无效会在启动前失败。异步步骤失败时记录 `FAILED` 和错误摘要；取消是轮询协作式，不是强制中断。应用重启会丢失内存中的运行任务，当前没有恢复 Worker。

10.6 关键代码位置

- `controller/SimulationSceneController.java`
- `controller/SimulationRunController.java`

- `service/impl/SimulationRunServiceImpl.java`：`startRun`
- `service/impl/SimulationRunExecutionServiceImpl.java`
- `service/impl/SimulationStepExecutionServiceImpl.java`
- `config/AsyncConfig.java`
- `resources/db/migration/V9__repair_simulation_model.sql`

11. 看板与后台任务流程

11.1 业务目标

提供低成本运营概览，并周期性维护离线状态和历史数据规模。

11.2 参与模块

Dashboard Controller/Service、资源和告警 Mapper、`CacheServiceImpl`、`DeviceMonitorTask`、`DataCleanupTask`、Redis、MySQL。

11.3 完整调用链

```
页面请求看板
  ↓
按当前组织/范围计算缓存键
  ↓
命中 Redis：直接返回
未命中：互斥锁 → 聚合 MySQL → 缓存 30 秒
```

后台任务按计划扫描设备最后在线时间生成旧离线告警，并按配置删除过期 `device_data`。

11.4 数据流

看板是派生读模型，不是事实源。缓存失效后可从数据库重建。后台清理删除旧固定字段遥测，但没有删除 `metric_sample`，导致新指标主链的保留策略尚未闭环。

11.5 异常路径

Redis 不可用时看板应回退数据库；互斥锁只减少缓存击穿，不保证强一致。定时任务失败会记录日志并等待下次执行，当前没有任务执行审计或补偿队列。

11.6 关键代码位置

- `controller/DashboardController.java`
- `service/impl/DashboardStatsServiceImpl.java`
- `service/impl/DashboardAlarmStatsServiceImpl.java`
- `task/DeviceMonitorTask.java`
- `task/DataCleanupTask.java`

CHAPTER

四、模块详细设计规范

本章要点

Controller 维护接口契约，Service 承担权限和事务，Mapper 只负责数据访问。跨模块直接操作 Mapper 会破坏当前正在形成的模块边界。

本章从内部实现视角统一描述核心模块。第二章回答“模块与谁协作”，本章回答“进入模块后代码如何组织”。

1. 账号认证模块

1.1 模块职责

业务目标是建立全局用户身份并安全签发登录凭证。设计上把平台角色放在用户主体，把组织角色放在成员关系，避免一个用户切换组织时重复签发 Token。边界不包含组织成员审批。

1.2 内部结构

```
AuthController / PlatformSetupController
  ↓
AuthService / PublicRegistrationService / PlatformSetupService
  ↓
SysUserMapper / PlatformRegistrationSettingsMapper
  + Redis / JWT / PasswordEncoder / Mail
  ↓
MySQL + Redis + QQ SMTP
```

1.3 数据模型设计

- `SysUser`：全局唯一 `username`、`email`，密码摘要，账号状态，`platformRole`；生命周期从注册/初始化开始，可被禁用但不随组织退出删除。
- `PlatformRegistrationSettings`：控制公开账号注册和组织申请入口，不承载付费套餐。
- Redis 验证码：短生命周期、按用途和邮箱绑定；仅保存摘要。
- JWT 黑名单：按 jti 或用户撤销时间存储，TTL 不超过 Token 剩余寿命。

1.4 核心接口设计

- `POST /api/auth/login`：输入登录标识和密码，输出 Token 与用户摘要。
- 公开验证码与注册接口：输入邮箱、用途、验证码和账号资料，成功后无敏感返回。
- `GET/POST /api/platform/setup`：读取初始化状态并一次性创建平台管理员。

1.5 实现方案

登录失败窗口用 Redis Lua 原子更新，避免并发请求绕过阈值。验证码使用 HMAC 摘要和原子消费，降低 Redis 泄露后直接利用验证码的风险。初始化采用环境 Bootstrap Token、邮箱所有权验证和数据库锁三重边界。

1.6 关键代码索引

`AuthController` 、 `AuthServiceImpl.login` 、 `PublicRegistrationServiceImpl.register` 、 `EmailVerificationServiceImpl` 、 `PlatformSetupServiceImpl.initialize` 、 `JwtUtil` 、 `SecurityConfig` 、V7 迁移。

2. 组织与成员模块

2.1 模块职责

业务目标是管理多组织关系和审批式组织开通。设计原因是账号注册不应强制创建组织，组织权限也不应永久写死在用户表。边界不包含机房级数据授权。

2.2 内部结构

```
Organization / CreationApplication / JoinApplication / Membership Controller
  ↓
申请 Service + MembershipService + ProvisioningService
  ↓
Organization / Membership / Application Mapper
  ↓
MySQL
```

2.3 数据模型设计

- `sys_organization`：组织代码和名称均唯一，代表租户边界。
- `organization_membership`：用户与组织的多对多关系，角色与状态在关系上；`UNIQUE(user_id, organization_id)`。
- 创建申请：审批通过前不产生正式组织。
- 加入申请：保存申请角色、审批人和处理状态。

成员关系是长期事实，前端“当前组织”只是请求上下文，不是数据库关系字段。组织管理员跨组织限制由 Service 规则维持。

2.4 核心接口设计

普通用户提交和查看自己的申请；平台管理员审核组织创建；目标组织管理员审核加入申请和维护成员；用户查询自己的活动组织列表。

2.5 实现方案

审批采用事务内状态检查。组织供应将组织与首个管理员一起创建。最后管理员保护采用加锁读取或条件校验，避免组织失去治理主体。拒绝、撤回和重复申请均通过有限状态管理。

2.6 关键代码索引

`OrganizationCreationApplicationServiceImpl` 、 `OrganizationJoinApplicationServiceImpl` 、 `OrganizationProvisioningServiceImpl` 、 `OrganizationMembershipServiceImpl` 、 `OrganizationContextFilter` 、V8 迁移。

3. 权限与数据范围模块

3.1 模块职责

业务目标是把认证身份转换为当前组织内的可操作数据集合。设计原因是 URL 角色无法表达“同为操作员但只能管理某些机房”。边界不负责账号认证和具体业务 CRUD。

3.2 内部结构

```
JWT SecurityContext + X-Organization-Id
  ↓
OrganizationContextFilter
  ↓
DataScopeService.resolveCurrentScope
  ↓
DataScopeContext
  ↓
各业务 Service 的 assert/canAccess 调用
```

3.3 数据模型设计

`sys_data_scope_assignment` 以用户、组织、机房和 `scope_type` 保存显式授权。 `DataScopeContext` 是运行期模型，包含范围类型、组织和机房 ID 集合，不持久化。

3.4 核心接口设计

`/api/auth/current-scope` 给前端展示当前范围；内部接口包括解析范围、检查组织、机房、设备读写能力。业务模块应依赖语义化方法，而不是直接解释 `cloudRoomIds`。

3.5 实现方案

组织管理员映射为 `ORG_ALL`，操作员有分配时映射为 `ROOMS`，监控员为 `READ_ONLY`。平台管理员不带组织头时是 `PLATFORM`。Service 在读取资源后检查其组织和机房，避免客户端提交伪造 ID。

3.6 关键代码索引

`OrganizationContextFilter` 、 `DataScopeServiceImpl.resolveCurrentScope` 、 `DataScopeContext` 、 `UserDataScopeServiceImpl` 、 `DataScopeViolationException` 、V2/V8 迁移。

4. 资源目录模块

4.1 模块职责

业务目标是维护稳定的资产主数据，为指标、告警和仿真提供引用。设计上将组织、机房、设备类型、设备分层，设备必须同时落在组织和机房。边界不接收高频时序数据。

4.2 内部结构

```
CloudRoom / DeviceType / Device Controller
  ↓
对应 Service + Excel Service
  ↓
Organization/DataScope 校验
  ↓
Mapper
  ↓
sys_cloud_room / device_types / devices
```

4.3 数据模型设计

- 机房属于组织；
- 设备属于组织、机房和设备类型；
- 设备编码承担业务识别，数据库 ID 承担关联；
- 状态与最后在线时间用于展示和离线判断；
- 删除设备类型前必须确认没有设备引用。

4.4 核心接口设计

机房和设备 CRUD、分页、详情、状态维护、Excel 模板/导入/导出。接口输入中的组织和机房标识必须与当前上下文一致。

4.5 实现方案

Service 先查权威资源再校验范围；导入按行解析并汇总错误，避免一行错误丢失全部诊断。设备类型使用受支持编码限制与下游测点/仿真能力保持一致。

4.6 关键代码索引

`CloudRoomServiceImpl`、`DeviceTypeServiceImpl`、`DeviceServiceImpl`、`DeviceServiceImpl.importFromExcelScoped`、`CloudRoomController`、`DeviceController`、V1/V2/V10 迁移。

5. 测点与指标模块

5.1 模块职责

业务目标是从固定字段遥测演进到可配置测点模型。类型模板表达默认能力，设备绑定表达最终生效配置，历史表和最新表分别服务趋势与状态查询。

5.2 内部结构

```
PointDefinition / Template / Binding / Metric Controller
  ↓
定义、模板、绑定、Sample、Latest、Retention Service
  ↓
设备与数据范围校验
  ↓
各 Mapper
  ↓
point_definition / templates / bindings / metric_sample / metric_latest
```

5.3 数据模型设计

- `PointDefinition`：测点语义、数据类型、单位、范围和状态；
- `DeviceTypePointTemplate`：设备类型默认测点；
- `DevicePointBinding`：设备实例有效配置；
- `MetricSample`：追加式历史事实；
- `MetricLatest`：每设备每测点一个快照；
- `MetricRetentionPolicy`：全局或测点级保留天数，最大 90 天。

历史和最新表不建立完整外键，降低高频写入耦合，但允许孤儿引用，必须由 Service 保证设备和测点有效。

5.4 核心接口设计

测点定义和模板由平台级管理；设备绑定由有权组织用户管理；指标写入接收设备、测点、值、质量和采样时间；查询提供历史分页/时间范围和最新值。

5.5 实现方案

写入前加载绑定和测点定义做类型/范围校验。历史先写入，再更新较新快照。VO 转换补充测点名称和单位。保留策略当前只有配置，没有执行器，是设计未闭环处。

5.6 关键代码索引

`PointDefinitionServiceImpl`、`DeviceTypePointTemplateServiceImpl`、`DevicePointBindingServiceImpl`、`MetricSampleServiceImpl`、`MetricLatestServiceImpl`、`MetricRetentionPolicyServiceImpl`、V3 迁移。

6. 旧遥测与规则告警模块

6.1 模块职责

业务目标是保持 v1 固定字段采集和告警功能可运行。它是兼容模块，不应继续承载新测点模型的全部需求。

6.2 内部结构

```
DeviceData Controller/Task
  ↓
DeviceDataServiceImpl
  ↓ 事务提交后
AlertRuleEngine
  ↓
Alert Mapper + NotificationDedup + Mail + Legacy WebSocket
```

6.3 数据模型设计

`device_data` 以固定列保存温度、湿度、电力等；`alerts` 保存设备级告警和状态；规则配置可在设备级覆盖类型级；`notification_log` 保存去重和结果。

6.4 核心接口设计

旧数据上报、查询、告警列表/处理、配置阈值。离线告警由定时任务而非用户接口触发。

6.5 实现方案

数据写入使用 `READ_COMMITTED` 并对死锁重试；规则评估在提交后执行；配置采用 cache-aside 和空值占位；通知失败与告警事实解耦。问题在于链路仍落 `alerts`，不会自动形成新 `alarm_event`。

6.6 关键代码索引

`DeviceDataServiceImpl`、`AlertRuleEngine`、`AlertConfigServiceImpl`、`NotificationDedupServiceImpl`、`EmailNotificationServiceImpl`、`AlertWebSocketService`、`DeviceMonitorTask`。

7. 权威告警与通知模块

7.1 模块职责

业务目标是提供统一告警生命周期和用户级通知。状态、转换和处理记录分表，避免只保留最终状态。边界不判断采集值是否越限。

7.2 内部结构

```
AlarmEventController
  ↓
AlarmEventServiceImpl（范围 + 状态机 + 事务）
  ↓
AlarmEvent/Transition/Handling Mapper
  ↓ 提交后
AlarmRecipientResolver
  ↓
InAppNotificationService + UserScopedWebSocketPublisher
```

7.3 数据模型设计

`alarm_event` 保存当前状态、严重度、资源坐标和版本；`alarm_state_transition` 保存状态变化；`alarm_handling_record` 保存人工动作；`in_app_notification` 以接收用户和来源保存可读状态。

7.4 核心接口设计

分页和详情查询、确认、进入处理、恢复、关闭、查看历史、通知列表/未读计数/标记已读。WebSocket 只发送事件类型、来源 ID 和更新时间等摘要。

7.5 实现方案

状态转换表定义允许边，条件更新用于检测并发状态变化，同一事务写当前状态和历史。提交后再投递通知，避免客户端收到尚未可查询的数据。接收人按组织和机房范围解析，但当前解析规则遗漏组织全量管理员的风险较高。

7.6 关键代码索引

`AlarmEventServiceImpl` 、 `AlarmRecipientResolverImpl` 、 `InAppNotificationServiceImpl` 、 `UserScopedWebSocketPublisher` 、 `AlarmEventController` 、V4/V6 迁移。

8. WebSocket 模块

8.1 模块职责

业务目标是低延迟通知前端“告警/通知发生变化”。它不是可靠消息队列，也不保存完整告警事实。

8.2 内部结构

```
/ws 握手
  ↓
WebSocketAuthenticationInterceptor（STOMP CONNECT JWT）
  ↓
SimpleBroker /queue + userDestinationPrefix /user
  ↓
/user/queue/summary
  ↓
前端 STOMP Client → REST 刷新
```

8.3 数据模型设计

WebSocket Session 由 Spring 进程内维护；Principal 使用用户标识；一个用户可有多个 Session；`WebSocketSummary` 是瞬时传输对象，不落库；站内通知才是可恢复记录。

8.4 核心接口设计

客户端只允许连接和订阅 `/user/queue/summary`，不允许向业务目的地 SEND。服务端通过 `convertAndSendToUser` 定向发送。

8.5 实现方案

HTTP 握手路径放行，认证在 STOMP CONNECT 帧执行，解决浏览器 WebSocket 握手头限制。前端设置心跳和 5 秒重连，收到摘要后重新请求 REST，保证显示数据以数据库为准。

8.6 关键代码索引

`WebSocketConfig` 、 `WebSocketHandshakeSecurityConfig` 、 `WebSocketAuthenticationInterceptor` 、 `WebSocketDestinations` 、 `UserScopedWebSocketPublisher` 、 `frontend/src/api/realtime.js` 。

9. 仿真模块

9.1 模块职责

业务目标是提供可审计、可取消、受权限控制的测试数据生成。设计上复用真实 Metric Service，而不是绕过规则直接插表。

9.2 内部结构

```
Scene/Run Controller
  ↓
SimulationSceneService / SimulationRunService
  ↓ 事务提交后
SimulationRunExecutionService（异步）
  ↓
SimulationStepExecutionService
  ↓
MetricSampleService + GeneratedDataRefMapper
```

9.3 数据模型设计

Scene 保存组织、机房和设备类型语境；Step 保存顺序、动作、测点和值；Run 保存场景快照、状态、进度和错误；GeneratedRef 保存运行与生成指标样本的逻辑关联。

9.4 核心接口设计

场景 CRUD、步骤维护、启动、查询运行、取消和查看生成引用。启动接口返回运行记录，不等待全部步骤完成。

9.5 实现方案

启动事务提交后才投递异步任务；CAS 防止重复启动；任务线程恢复组织上下文和 MDC；步骤执行周期性检查取消；生产 profile 和写入开关形成双重安全保护。

9.6 关键代码索引

`SimulationSceneServiceImpl`、`SimulationRunServiceImpl.startRun`、`SimulationRunExecutionServiceImpl`、`SimulationStepExecutionServiceImpl`、`AsyncConfig`、V9 迁移。

10. 横切基础设施模块

10.1 模块职责

业务目标是提供统一返回、异常、日志、审计、缓存、限流和数据库迁移。其边界是技术一致性，不能吞掉业务事实或重新解释业务权限。

10.2 内部结构

Filter/Interceptor/AOP 位于入口；`GlobalExceptionHandler` 位于异常出口；`CacheServiceImpl`、Mail、Mapper 和 Flyway 位于基础设施适配层。

10.3 数据模型设计

`sys_audit_log` 保存用户、操作、IP、详情、结果和 trace 字段；`notification_log` 保存外部通知状态。缓存键是派生状态，TTL 到期后必须能从数据库重建。

10.4 核心接口设计

`Result<T>` 是 HTTP body 契约；业务异常携带 code/message；`CacheService` 区分普通缓存和必须原子的计数操作；`@AuditLog` 标记需要记录的控制器动作。

10.5 实现方案

审计切面无论成功失败都尝试写日志，审计自身失败不覆盖原业务异常。缓存读取失败对数据库可回源场景静默降级，限流和安全计数则使用更严格策略。Flyway 采用追加式迁移并保留修复脚本。

10.6 关键代码索引

`Result` 、 `ResultCode` 、 `GlobalExceptionHandler` 、 `AuditLogAspect` 、 `TraceIdFilter` 、 `RateLimitFilter` 、 `CacheServiceImpl` 、 `MybatisPlusConfig` 、 `resources/db/migration/` 。

CHAPTER

五、异常处理机制分析

本章要点

HTTP、Filter、WebSocket 和异步线程有不同异常出口。统一错误码、traceId 和结构化审计，是把这些出口重新关联起来的关键。

1. 异常分类

当前异常可按来源分为：

类型	典型来源	处理目标
参数校验异常	Bean Validation、JSON 反序列化、类型转换	返回明确字段或格式错误，不进入业务写入
认证异常	JWT 缺失、过期、黑名单、STOMP CONNECT 失败	返回 401 或拒绝 WebSocket 连接
授权异常	Spring Security 角色不足	返回 403，并记录访问拒绝审计
数据范围异常	组织、机房、设备不在当前授权范围	返回 403 语义的业务错误
业务异常	重复、非法状态、最后管理员、验证码错误	返回可预期业务消息
资源不存在	用户、组织、设备、事件等未找到	返回 404 语义
缓存异常	Redis 读取、写入、原子计数失败	可回源场景降级；安全计数场景显式失败
数据异常	唯一键、死锁、SQL、事务冲突	转换为业务冲突或系统错误；死锁局部重试
系统异常	未预期运行时错误、外部服务故障	记录 trace 和堆栈，返回通用 500

2. 异常产生位置

- Filter：Token、限流、XSS 包装和组织上下文前置问题；
- Controller：参数校验、请求体反序列化、接口级角色校验；
- Service：绝大多数业务规则、数据范围、状态机和一致性异常；
- Mapper/数据库：唯一键、连接、SQL、锁超时和死锁；
- 基础设施适配器：Redis、SMTP、WebSocket、异步线程池；
- 定时任务和异步任务：没有 HTTP 调用栈，需要自行记录并更新任务状态。

设计上应优先在最理解语义的 Service 产生业务异常。例如设备存在关联时应抛“设备类型被引用”的领域异常，而不是让外键异常泄漏到 Controller。

3. 异常传播流程

```
Controller/Service/Mapper 产生异常
  ↓
事务拦截器判断回滚
  ↓
@AuditLog 切面记录失败（不覆盖原异常）
  ↓
GlobalExceptionHandler 按异常类型映射
  ↓
Result<T> + 对应 HTTP 状态
  ↓
Axios 响应拦截器显示消息、处理 401/403
```

Security Filter 阶段的异常不一定经过 `GlobalExceptionHandler`，因此 `AuthenticationEntryPoint`、`AccessDeniedHandler` 和限流过滤器需要自行构造一致响应。WebSocket CONNECT 异常则由 STOMP 通道拒绝，不使用普通 REST Result。

异步仿真没有 Controller 调用栈，异常由执行服务捕获后写入 Run 的 `FAILED` 状态和错误信息，同时记录 trace 日志。通知属于提交后的副作用，失败不应回滚已经成立的告警事实。

4. 全局异常处理

`GlobalExceptionHandler` 已提供统一处理，覆盖校验、业务、未找到、数据范围、缓存、访问拒绝和通用异常。返回体使用 `Result<T>` 的 `code/message/data`。`TraceIdFilter` 将追踪标识放入 MDC，审计表也包含结构化字段，便于把接口错误与操作记录关联。

当前错误码体系处于过渡状态：`ResultCode` 已出现稳定字符串代码的方向，但 `Result` 仍主要使用整数 code，部分业务异常还直接使用 400/403 等数值。HTTP status、body code 和业务语义没有完全统一。

日志策略的合理部分：

- 预期业务拒绝通常记录 WARN，不需要完整堆栈；
- 未预期系统异常记录 ERROR 和堆栈；
- 审计记录操作者、IP、操作和结果；
- 异步线程通过 MDC 装饰器传播 trace。

5. 当前异常处理问题

1. **错误码双轨**：稳定业务 code 与 HTTP 数字 code 并存，前端难以建立长期分支逻辑。
2. **Filter 响应不完全一致**：限流使用 HTTP 429，但手工 body 可能仍写 `code: 400`。
3. **通用异常过多**：如设备类型关联校验使用 `IllegalStateException`，丢失明确领域类型。
4. **组织上下文错误不够直接**：无效组织头或无效成员可能被过滤器静默忽略，最终在更深层表现为“缺少上下文”。

5. **审计详情非结构化**：注解模板做简单字符串替换并依赖 DTO `toString()`，有敏感字段泄露和后续检索困难风险。
6. **异步任务恢复不足**：进程重启后 `RUNNING` 仿真没有统一恢复/失败收敛机制。
7. **外部通知失败可观测性有限**：虽有 notification log，但缺少统一重试队列、告警和运维视图。

CHAPTER

六、关键设计决策分析

本章要点

当前主要设计都服务于低成本一致性：单库本地事务、Redis 短期状态、历史与最新双表、摘要推送加 REST 回源。局限主要出现在并发、集群和模型过渡。

1. 全局账号 + 组织成员关系

背景问题

用户可能加入多个组织，平台角色与组织角色语义不同，把 `organization_id` 和角色直接写在用户表会导致切换、审批和历史关系难以表达。

当前方案

`sys_user` 保存全局身份和可选平台角色；`organization_membership` 保存用户在组织中的角色和状态；请求头选择当前组织。

设计原因

身份生命周期通常长于组织关系，且一个账号需要复用邮箱、密码和安全策略。关系表也便于记录审批、角色变化和活动状态。

优点

模型清晰、支持多组织普通成员、Token 无需随组织切换重签、组织审批可独立演进。

局限

每个组织请求都要解析成员关系；旧用户表中的 `role/organization_id` 字段仍可能造成理解混淆；跨组织管理员约束依赖事务业务规则。

替代方案

每组织独立账号会简化授权但造成重复身份和邮箱；JWT 内放全部组织角色会使 Token 膨胀且角色变更不能及时生效。当前方案更适合项目现阶段。

2. 双层权限：入口 RBAC + Service 数据范围

背景问题

单靠接口角色不能表达机房级授权，单靠前端隐藏按钮也无法防止直接请求。

当前方案

Spring Security 控制接口级角色；`OrganizationContextFilter` 建立组织角色；Service 使用 `DataScopeService` 检查具体组织、机房、设备。

设计原因

粗粒度判断越靠前越节省资源，细粒度判断必须在加载权威数据后完成。内部调用和异步任务也需要复用 Service 校验。

优点

纵深防御、权限语义可复用、避免 Controller 堆积资源校验。

局限

同一规则可能在注解、过滤器和 Service 多处维护；范围辅助方法语义不统一时容易错误拦截；平台管理员跨组织策略在不同模块不完全一致。

替代方案

可引入统一 Policy 对象或方法级授权表达式，但当前不需要外置策略服务。优先把 `canRead/canWriteOrganization/CloudRoom/Device` 语义收敛到一个接口。

3. MySQL 单库 + Flyway 追加迁移

背景问题

当前业务跨账号、组织、资源和审批有大量本地事务，需要稳定快速迭代数据库模型。

当前方案

单 MySQL schema，MyBatis-Plus 访问，Flyway V1-V10 追加迁移，修复历史数据使用新版本脚本。

设计原因

单库事务能低成本保证组织供应、成员审批、告警状态历史等一致性。追加迁移可复现每个已部署环境的升级路径。

优点

部署简单、事务明确、查询方便、适合当前规模。

局限

模块没有数据所有权，迁移耦合；高频指标与管理表共享数据库；部分逻辑引用缺少外键。

替代方案

现阶段不建议直接分库。可先按表前缀/包边界明确所有权，遥测增长后再评估时序数据库和异步事件桥。

4. Redis 用于短期安全状态和缓存

背景问题

验证码、冷却、失败窗口、限流、Token 撤销和热点看板不适合每次都写 MySQL。

当前方案

Redis 保存 TTL 状态、Lua 原子计数、黑名单、cache-aside 数据和互斥锁。

设计原因

Redis 原子操作与过期机制适合短生命周期数据，可降低数据库压力。

优点

实现简单、响应快、天然 TTL、支持滑动窗口和锁。

局限

Redis 故障时不同功能的降级安全等级不同；键规范和 TTL 是隐性契约；单 Redis 仍可能是可用性瓶颈。

替代方案

验证码和限流可用专门网关或风控服务，但现阶段收益不足。关键是明确哪些功能 fail-open、哪些 fail-closed，并为 Redis 故障做测试。

5. 历史指标 + 最新快照双表

背景问题

趋势查询需要历史数据，设备状态页只需要最新值。每次从历史表取最大时间会随数据量增长变慢。

当前方案

每次写 `metric_sample` 后维护 `metric_latest`，只接受采样时间更新的数据。

设计原因

以写放大换取稳定低延迟读取，符合监控系统读多且状态页频繁刷新的特点。

优点

最新状态查询简单；历史与快照职责明确；仿真和真实采集可复用同一接口。

局限

当前读后写更新不是完全原子；两表一致性依赖事务；历史清理未实现；无外键可能产生孤儿数据。

替代方案

MySQL 可使用单条 `INSERT ... ON DUPLICATE KEY UPDATE` 加时间条件；更大规模可用时序数据库 + 独立最新值缓存。

6. 两套遥测/告警模型并存

背景问题

基础项目使用固定字段 `device_data` 和 `alerts`，产品化方向引入通用测点和可审计 `alarm_event`，不能一次性删除旧功能。

当前方案

保留旧链运行，同时新增测点指标和告警事件模型。

设计原因

降低一次性重构风险，让前端和业务逐步迁移。

优点

旧功能可继续演示，新模型可以独立开发和测试。

局限

同一设备可能有两套事实；看板、通知和 WebSocket 口径可能分裂；当前没有生产桥接，`alarm_event` 状态机可能无数据。

替代方案

推荐过渡方案不是长期双写，而是定义统一 `AlarmTriggered` 领域事件：旧规则引擎和未来测点规则都发布事件，由告警模块幂等创建 `alarm_event`，随后迁移查询并停止写旧 `alerts`。

7. WebSocket 摘要 + REST 回源

背景问题

告警变化需要即时反馈，但通过 WebSocket 发送完整 VO 会复制 REST 契约、增加权限和断线恢复复杂度。

当前方案

WebSocket 只向目标用户推送 `WebSocketSummary`，前端收到后调用 REST 获取权威数据。

设计原因

数据库和 REST 仍是事实源，实时通道只负责失效通知。

优点

消息体小、前后端低耦合、断线后可通过 REST 恢复、权限过滤集中在查询接口。

局限

会产生额外 REST 请求；简单 Broker 仅单机；无持久化和重放；快速事件可能触发重复刷新。

替代方案

短期可在前端做刷新合并；多实例时使用外部 STOMP Broker 或 Redis/Kafka 事件广播，但站内通知仍需落库。

8. 仿真复用正式指标写入

背景问题

若仿真直接插表，可能绕过校验、最新值更新和权限，得到与真实采集不同的结果。

当前方案

仿真步骤调用 `MetricSampleServiceImpl`，并记录生成引用。

设计原因

同一写入路径能验证真正业务能力，也减少重复逻辑。

优点

数据格式一致、权限一致、最新值同步、可追踪来源。

局限

仿真与生产数据混表；清理策略不足；异步执行只在进程内；若告警桥接缺失，无法验证完整闭环。

替代方案

当前阶段方案合理。后续可增加 `source_type`/环境隔离，或让外部仿真器调用正式采集 API。

CHAPTER

七、WebSocket 实时通知专项分析

本章要点

WebSocket 只承担变化通知，站内通知和告警事件承担可恢复事实。多实例前必须解决 Session 和 Broker 的跨节点传播。

1. 使用场景

告警状态和站内通知变化具有时效性。轮询会在无变化时浪费请求，并增加用户看到延迟。WebSocket 用来提示目标用户“某类资源已变化”，前端再拉取权威数据。

2. 生命周期

2.1 建立连接

前端通过 SockJS/STOMP 连接 `/ws`。HTTP 握手路径在 Spring Security 中允许访问，因为浏览器握手阶段不便统一设置标准 Authorization 头。

2.2 身份认证

客户端在 STOMP `CONNECT` 帧携带 Bearer JWT。`WebSocketAuthenticationInterceptor` 校验 Token、黑名单和用户状态，并把用户标识设置为 Principal。握手放行不等于匿名订阅。

2.3 用户绑定与订阅

服务器配置 `userDestinationPrefix=/user` 和简单 Broker 前缀 `/queue`。客户端只订阅 `/user/queue/summary`；服务端调用 `convertAndSendToUser` 时，Spring 根据 Principal 把消息路由到该用户的所有有效 Session。

2.4 Session 管理

Session 由当前 Spring Boot 进程内存维护。一个用户可同时在多个浏览器或标签页连接，服务器会向其多个 Session 投递。当前没有业务层持久化 Session 表。

2.5 心跳与重连

前端配置约 10 秒心跳和约 5 秒重连。心跳用于发现断线，重连后重新订阅。断线期间错过的摘要不会重放，但未读站内通知和告警事实可由 REST 恢复。

2.6 断开处理

断开由 STOMP 客户端和 Spring Session 生命周期清理。当前没有显式在线用户业务状态，也没有将断开事件写审计表。

3. 消息流程

```
AlarmEvent 状态事务提交
  ↓
AlarmRecipientResolverImpl 解析用户 ID
  ↓
InAppNotificationServiceImpl 落库
  ↓
UserScopedWebSocketPublisher
  ↓
SimpMessagingTemplate.convertAndSendToUser
  ↓
/user/queue/summary
  ↓
前端 websocket.js
  ↓
刷新告警列表、未读数量或详情 REST
```

使用事务提交后发送很重要：如果在事务内发送，客户端可能收到消息后立即查询，却读不到未提交状态；事务最终回滚时还会产生虚假通知。

4. 数据关系

- 用户：来自 `sys_user.id`，也是 Principal 路由键；
- Session：一个用户对应零到多个当前连接；
- 连接：瞬时网络状态，不是业务事实；
- `in_app_notification`：持久化的用户级通知，可读、可追踪；
- `WebSocketSummary`：瞬时变化信号，可丢失但可由 REST 恢复；
- `alarm_event`：权威告警事实和状态。

5. 当前限制

1. Simple Broker 和 Session 都在单实例内，多实例部署时用户连接可能收不到另一实例产生的消息。
2. 摘要不持久化、不重放，依赖站内通知和 REST 恢复。
3. 旧 `AlertWebSocketService` 发送 `/topic/alerts`，但 Broker 只启用 `/queue`，前端也未订阅该 topic，属于失效兼容代码风险。
4. 接收人解析当前主要依赖显式机房授权，可能漏发没有 assignment 的 `ORG_ADMIN`。
5. 前端每条摘要都触发刷新时可能形成请求风暴，应合并短时间内相同资源刷新。
6. JWT 存在 `localStorage`，若发生前端 XSS，WebSocket 和 REST Token 都可能被窃取。

CHAPTER

八、工程经验专题

本章不引入新的项目事实，而是从当前实现提炼可迁移的工程方法。阅读时应把“当前实现”与“推荐演进”分开：前者可从代码直接验证，后者是基于现状的改进路径。

适合重点阅读的人
维护认证、权限、缓存、实时通知、数据库和异步任务的开发者。

阅读目标
理解方案为什么成立、在哪些边界会失效，以及扩展前应先补哪项能力。

事实基线
Spring Security + JWT、Redis、MySQL、STOMP Simple Broker、进程内异步执行器。

判断原则
优先保证事实唯一、权限一致、事务边界明确和故障后可恢复。

1. Spring Security 与组织数据权限

设计思考卡片

背景	系统同时存在平台角色、组织角色和机房级授权。
问题	单纯依赖 URL 角色或前端菜单，无法判断用户是否能操作某一条机房、设备或指标数据。
方案	使用 JWT 建立全局身份，OrganizationContextFilter 建立当前组织，Service 再执行资源数据范围校验。
收益	入口能快速拒绝明显越权，内部调用和异步任务也能复用同一业务权限规则。
经验	认证回答“你是谁”，组织上下文回答“你代表哪个组织”，数据范围回答“你能处理哪条数据”。三者不能合并成一个 role 字符串。

设计思想

将粗粒度 RBAC 放在 Filter/Security 层，将资源级 ABAC 放在 Service 层。JWT 只携带变化较慢的全局身份，不固化组织角色，避免成员关系变化后 Token 长时间过期失真。

工程技巧

- Filter 顺序应固定：追踪和限流在前，JWT 建立身份，组织过滤器在身份之后；
- Service 使用 `ensureDeviceAccess`、`canAccessCloudRoom` 等语义化接口，不直接解释 ID 集合；
- 后台任务和异步线程没有 HTTP 请求上下文，必须显式建立或传播 CurrentOrganizationContext；
- 权限拒绝需要记录用户、组织、资源类型、资源 ID 和 traceId，但不要记录 Token。

常见问题

- 把组织角色写入 JWT，导致角色变更不能即时生效；
- 前端隐藏按钮后认为接口已经安全；
- `ORG_ALL` 用空机房集合表示，却被通用 `contains` 判断成无权限；
- 平台管理员是否允许代维没有统一策略，导致不同模块行为不一致。

优化方向

建立统一资源 Policy：分别提供组织、机房、设备的读写判断；为每类范围补参数化测试；把“无组织头”“成员无效”“资源越权”转换为不同稳定错误码。

2. Redis 缓存与短期安全状态

技术方案卡片

技术	Redis + TTL + Lua 原子脚本 + cache-aside。
作用	验证码、发送冷却、登录失败窗口、Token 黑名单、接口限流、热点缓存和互斥锁。
使用方式	短期状态写 Redis，长期业务事实写 MySQL；可回源缓存允许降级，安全计数按风险决定 fail-open 或 fail-closed。
设计注意事项	缓存键、TTL、原子性、空值策略和故障降级必须作为接口契约管理。

设计思想

Redis 最适合保存“可过期、可重建或需要原子计数”的状态，不应成为组织、权限、告警状态等永久事实源。验证码只缓存 HMAC 摘要，体现了即使缓存泄露也减少明文凭证暴露的设计。

工程技巧

- cache-aside 写路径先更新数据库，再删除或更新缓存；
- 用空值占位抑制不存在 key 的重复回源；
- 登录失败和验证码配额使用 Lua，把判断和更新放入一个原子操作；
- 缓存键中包含组织、用户或范围版本，避免不同权限共享结果；
- 互斥锁只减少缓存击穿，不应承担业务唯一性。

常见问题

缓存与数据库短时不一致、热点 key、TTL 同时到期、Redis 故障策略不明确、锁未设置过期时间、把限流失败统一放行造成安全边界降低。

优化方向

建立缓存键注册表和指标监控；对安全相关 Redis 操作增加故障演练；为看板缓存加入随机抖动；必要时使用本地短缓存降低 Redis 网络依赖。

3. WebSocket 长连接与用户级通知

设计思考卡片

背景	告警变化需要低延迟触达，但完整告警对象已经由 REST 提供。
问题	在 WebSocket 中复制完整业务 VO，会产生权限、版本、断线恢复和消息可靠性问题。
方案	持久化站内通知，WebSocket 只向目标用户发送摘要，客户端收到后 REST 回源。
收益	实时通道与数据契约低耦合，消息丢失后仍能从数据库恢复。
经验	实时消息可以是“缓存失效信号”，不必承担业务事实传输。

设计思想

连接、Session 和消息摘要是瞬时状态，`alarm_event` 与 `in_app_notification` 才是事实。用户与 Session 是一对多关系，`convertAndSendToUser` 通过 Principal 将同一用户的多个标签页连接统一路由。

工程技巧

- 在 STOMP CONNECT 帧验证 JWT，不把握手放行误解为匿名访问；
- 限制客户端 SEND 目的地，只允许订阅受控用户队列；
- 使用心跳发现半开连接，重连后重新订阅；
- 对短时间内多个摘要做前端刷新合并；
- 事务提交后再推送，避免客户端读到未提交或回滚数据。

常见问题

单机 Session 无法跨实例、Simple Broker 不持久化、旧 `/topic/alerts` 与新 `/queue` 配置冲突、接收人解析漏掉组织管理员、断线期间消息无法重放。

优化方向

多实例部署前引入外部 Broker 或事件广播；保持站内通知落库；建立未读游标或增量同步接口；清理旧 topic 兼容代码；为接收人策略建立组织角色测试矩阵。

4. MySQL 数据模型、索引与查询

技术方案卡片

技术	MySQL + MyBatis-Plus + Flyway + 本地事务。
作用	保存账号、组织、资源、指标、告警、通知、仿真和审计的权威事实。
使用方式	管理类数据使用规范化关系模型；指标使用历史追加表和最新快照表；状态机使用当前状态加不可变历史。
设计注意事项	索引必须从真实过滤、排序和关联路径出发；无外键表必须由 Service 和一致性任务补偿。

设计思想

单库本地事务适合当前规模，能够低成本保证审批、组织供应和告警状态历史一致。`metric_sample` 与 `metric_latest` 分离是典型的写放大换读性能：趋势查历史，状态页查快照。

工程技巧

- 联合索引按等值过滤列、范围列和排序列安排，不能只给每列单独建索引；
- 分页限制最大 size，避免无界查询；
- 审批和最后管理员保护使用 `FOR UPDATE` 或条件更新；
- 最新值使用带时间条件的原子 upsert，正确处理乱序数据；
- Flyway 迁移只追加，新修复写新版本，不修改已执行脚本；
- 高频列表将逐条补充查询改为批量加载或 join，避免 N+1。

常见问题

索引与 SQL 条件顺序不匹配、逻辑删除字段遗漏、`IN` 列表过大、读后写竞态、历史表无清理、删除元数据后留下孤儿指标。

优化方向

先实现 `metric_sample` 保留策略和原子 latest upsert；用慢查询与执行计划驱动索引调整；数据量确实超过 MySQL 适用范围后，再评估时序数据库，而不是提前拆分。

5. 事务提交后副作用与异步任务

经验总结区域

可迁移经验 数据库事务只保证本地事实；邮件、WebSocket 和异步执行属于外部副作用，应在提交后触发，并具备幂等、失败记录和恢复策略。

项目实例 旧规则评估在遥测提交后执行；告警通知在状态提交后发送；仿真任务在 Run 记录提交后进入线程池。

设计思想

先持久化“发生了什么”，再执行“通知谁、调用谁”。这样外部失败不会抹掉业务事实，客户端也不会看到最终回滚的数据。

工程技巧

- 使用事务同步回调或 outbox 表表达提交后事件；
- 异步任务启动前保存快照和 PENDING 状态；
- 用 CAS 防止同一任务重复启动；
- 线程池有界，并传播 MDC、用户和组织上下文；
- 取消采用协作式检查，运行状态必须最终收敛；
- 外部副作用记录幂等键、尝试次数、最后错误和下次重试时间。

常见问题

在事务内发邮件导致长事务；提交前推送造成脏读；应用重启后 RUNNING 永久悬挂；重试没有幂等产生重复通知；线程池无界拖垮主进程。

优化方向

当前阶段可先增加启动时任务收敛和通知重试表。进入多实例后，再引入 outbox + 消息队列，不应直接把本地回调替换为不可靠的“发送后即忘”。

本章要点

认证、缓存、数据库、实时连接和异步任务看似是基础设施问题，实际都围绕同一原则：事实必须唯一且可恢复；权限必须在权威数据处校验；外部副作用不能破坏本地事务。

CHAPTER

九、性能分析与系统演进

本章要点

当前先做单体优化和边界强化；只有出现独立扩缩容、数据技术或发布节奏需求时，才有足够理由拆分服务。

1. 当前已实现的优化

1.1 缓存

- 字典、告警配置采用 cache-aside，并使用空值占位降低缓存穿透；
- 看板按组织/范围缓存约 30 秒，并通过 Redis 互斥锁降低并发回源；
- Redis 故障时，数据库可回源的读场景选择降级；
- 登录失败、验证码、限流使用 Lua/原子计数，避免读改写竞态。

1.2 SQL 与分页

- MyBatis-Plus 分页插件限制最大页大小为 100，防止无界列表；
- 迁移已为组织、设备、指标、告警和申请的主要查询字段建立索引；
- `metric_latest` 用唯一键提供最新状态快速查询；
- 关键审批和最后管理员规则使用锁；
- 仿真运行使用条件更新实现状态 CAS。

1.3 事务与异步

- 旧遥测写入对数据库死锁做有限重试；
- 规则评估、告警通知和仿真投递尽量安排在事务提交后；
- 仿真使用独立线程池（核心/最大线程和队列均有界）并传播 MDC；
- 通知失败与核心业务事实解耦。

1.4 数据读模型

- 历史指标与最新快照分离；
- 看板聚合结果短期缓存；
- 告警当前状态与转换历史分离；
- WebSocket 只发摘要，减少序列化和网络负载。

2. 当前性能限制

1. `MetricLatest` 的读后插入/更新在高并发同一设备测点写入时可能竞争，建议改为带时间条件的数据库原子 upsert。
2. Metric VO 补充测点名称等信息时存在逐条查询风险，历史列表可能形成 N+1。
3. 部分旧告警分页会先计算完整授权设备 ID 集合，组织设备量增大后内存和 SQL `IN` 压力明显。
4. 看板聚合仍可能把较多记录加载到 JVM 后计算，应逐步下推为数据库聚合或专用读模型。
5. `metric_sample` 没有执行中的清理任务，数据会持续增长，最终影响索引、备份和查询。
6. Simple STOMP Broker、异步仿真线程池和定时任务均是单进程能力，不能水平扩展。
7. 邮件发送是外部慢调用，若同步位于用户请求链会增加尾延迟；应确认各调用点均避免阻塞核心事务。
8. 审计每次敏感操作都写数据库，在高频采集接口上不应滥用 `@AuditLog`。

3. 架构演进方向

阶段一：单体优化

适用于当前版本，重点不是拆服务，而是消除模型断链：

- 建立指标/规则到 `alarm_event` 的唯一生产入口；
- 统一数据范围语义，修复 `ORG_ALL` 和接收人解析；
- 实现 `metric_sample` 保留策略执行；
- 将 MetricLatest 改为原子 upsert；
- 批量加载 VO 依赖，消除 N+1；
- 统一错误码和 Filter/Controller 响应；
- 给仿真运行增加应用重启后的状态收敛；
- 明确旧 `device_data/alerts` 退役路径。

阶段二：模块边界强化

仍可维持一个部署单元，但要求代码依赖可控：

- 账号、组织、资源、遥测、告警、仿真分别定义 facade 和内部包；
- 禁止跨模块直接调用 Mapper；
- 用领域事件接口表达 `MetricAccepted`、`AlarmTriggered`、`AlarmStateChanged`；
- 为每个模块定义拥有的表和只读引用；
- 用架构测试限制包依赖；
- 将旧兼容链包装在明确 adapter 中。

这一步完成后，系统才真正接近模块化单体。

阶段三：按压力微服务演进

只有出现独立扩缩容、部署节奏或数据技术需求时再拆分：

候选服务	拆分动因	通信方式	主要成本
遥测采集/指标服务	高频写入、历史数据量、时序数据库	异步事件 + 查询 API	元数据缓存、幂等、乱序、数据迁移
告警服务	状态机、通知量、独立扩展	消费 Metric/AlarmTriggered 事件	可靠消息、事件去重、权限快照
身份服务	多产品复用、安全独立治理	JWT/OIDC + 用户 API	角色同步、Token 撤销、可用性
通知服务	邮件/短信/站内/推送渠道扩展	可靠消息队列	模板、重试、死信、用户偏好
仿真 Worker	大规模场景、隔离生产线程	任务队列	任务恢复、取消协议、结果回传
看板读模型	聚合压力和多源数据	事件投影 + 查询 API	最终一致、重建、口径治理

微服务化需要同时解决：

- 服务通信：同步查询只用于低频元数据，事实变化优先事件；
- 数据隔离：每个服务拥有自己的写模型，禁止跨库直查；
- 分布式事务：使用 outbox、幂等消费者和补偿，不复制本地大事务；
- 服务治理：配置、追踪、监控、熔断、重试和部署自动化；
- 权限：组织和数据范围需要可验证的声明或低延迟策略缓存。

在这些能力不存在前，拆服务会降低可靠性而不是提升架构质量。

CHAPTER

十、技术债与设计问题分析

本章要点

P0 问题直接影响告警闭环和权限正确性；P1 影响事实一致性与长期运行；P2 适合纳入持续治理。

以下结论只做分析，本文未修改业务代码。优先级依据“是否阻断业务闭环、是否造成越权/漏权、是否导致事实不一致”排序。

P0：应在下一轮功能开发前处理

1. 新告警事件没有生产创建入口

现状：`AlarmEventServiceImpl` 实现了查询和状态转换，测试可直接准备事件，但生产代码搜索不到从指标、规则或旧 `AlertRuleEngine` 创建 `alarm_event` 的调用链。

影响：新版告警页面、站内通知和用户级 WebSocket 可能没有真实业务数据；仿真写入指标也无法验证完整告警闭环。

建议方向：定义幂等 `AlarmTriggered` 入口，由规则引擎发布，告警模块创建/更新 `alarm_event`。不要让多个调用方直接操作 Mapper。

2. `ORG_ALL` 的机房访问语义存在误判

现状：`DataScopeContext.hasCloudRoomAccess` 主要检查 `cloudRoomIds`，而 `ORG_ALL` 通常不展开全部机房 ID。`DataScopeServiceImpl.canAccessCloudRoom` 和 `AlarmEventServiceImpl.pageEvents` 等代码存在直接使用该帮助方法的情况。

影响：组织管理员明明拥有全组织权限，却可能收到“无权访问 CloudRoom”或查询不到告警。此前仿真模块已通过资源服务校验绕开同类问题，但问题没有全局收敛。

建议方向：所有资源判断统一调用理解 `PLATFORM/ORG_ALL/ROOMS/READ_ONLY` 的语义化 Policy 方法，禁止业务代码直接检查 ID 集合。

3. 告警接收人可能漏掉组织管理员

现状：`AlarmRecipientResolverImpl` 主要从 `sys_data_scope_assignment` 查精确机房授权，并参考用户表旧角色字段。`ORG_ADMIN` 通常不需要显式 assignment，新全局账号的旧 `role` 字段也可能仍为 `VIEWER`。

影响：组织管理员可以在页面看到告警，但收不到站内通知和 WebSocket 摘要，形成难以察觉的漏通知。

建议方向：接收人应以 `organization_membership` 的活动角色为主：组织管理员全组织接收，操作员按机房 assignment，监控员按产品策略决定只读通知。

P1：影响一致性、可维护性或中期运行

4. 新旧遥测和告警事实分裂

`device_data/alerts` 与 `metric_sample/metric_latest/alarm_event` 同时存在，没有明确双写、迁移、查询口径和退役时间。设备离线任务仍写旧告警，看板可能混用不同来源。任何新增功能都必须先明确落在哪条链。

5. `metric_sample` 保留策略未执行

`MetricRetentionPolicyServiceImpl` 和表结构存在，但 `DataCleanupTask` 只清理 `device_data`。保留天数当前是配置而不是实际行为，长期会造成数据库持续增长。

6. 旧 WebSocket 目的地与当前配置冲突

`AlertWebSocketService` 广播 `/topic/alerts`，`WebSocketConfig` 只启用 `/queue`，前端订阅 `/user/queue/summary`。旧广播属于未完成迁移或无效代码，容易让维护者误以为告警已实时推送。

7. MetricLatest 并发更新不是数据库原子操作

先查后插入/更新在并发写入同一设备测点时可能发生唯一键冲突或较旧值覆盖窗口。应使用单 SQL upsert，并在 `sample_time` 更新条件中处理乱序。

8. 平台管理员业务穿透策略不一致

资源模块倾向阻止平台管理员直接操作组织业务，告警和仿真部分路径允许平台管理员在指定组织上下文后跨组织操作。若这是运维能力，应形成明确“平台代维模式”；若不是，应统一禁止。

9. 仿真运行缺少重启恢复

异步任务存在于进程线程池。应用重启后，数据库中 `PENDING/RUNNING` 记录不会自动继续或标记失败，用户可能长期看到悬挂任务。

10. 指标与告警部分表缺少外键

这是写入性能与演进灵活性的主动取舍，但删除设备/测点后可能留下 `metric_sample`、`metric_latest`、`alarm_event` 和仿真引用。需要明确归档/软删除规则和定期一致性检查。

P2：应在持续维护中收敛

11. 错误码体系尚未统一

`ResultCode` 的稳定 code 方向与 `Result` 整数 code 并存；HTTP status 与 body code 偶有不一致。前端只能依赖 message 或多个数字分支，国际化和自动化处理困难。

12. 审计详情可能包含敏感字段

`@AuditLog` 详情模板和 DTO `toString()` 缺少结构化字段白名单与脱敏。登录、注册、Token、邮箱验证码相关 DTO 必须确认不会进入明文日志。

13. 前端 Token 存储在 localStorage

任何成功的前端 XSS 都可能读取 JWT。后端 `XssFilter` 不能覆盖 DOM 注入和第三方依赖风险。后续可评估 HttpOnly Cookie + CSRF 防护，至少先落实 CSP、依赖审计和输出编码。

14. VO 转换和授权列表可能产生 N+1/大 IN

指标列表逐项补测点信息、旧告警先加载全部设备 ID 等实现，在当前数据量可运行，但组织规模扩大后需要批量查询和 SQL join/exists。

15. 乐观锁插件未形成统一保护

MyBatis-Plus 注册了乐观锁拦截器，但多数 Entity 没有 `@Version`。告警和仿真使用手工条件更新，其他更新仍可能后写覆盖前写。应按模块明确并发策略，而不是认为“已配置插件”就自动安全。

16. 组织上下文失败信息不够精确

过滤器对非法 header、无成员关系等情况若选择不建立上下文，深层 Service 最终只报告缺少组织。这增加排障成本，也可能让前端无法区分“请选择组织”和“无权加入该组织”。

17. 本地集成测试与开发数据库隔离需要持续检查

CI 已提供独立 MySQL/Redis，但本地 `SpringBootTest` 若默认加载 dev profile，仍可能触碰开发数据。测试 fixture 曾发生污染修复，说明需要显式 test profile、随机 schema 或容器化数据库。

18. 迁移历史包含兼容修复复杂度

V1 与 V6 对通知日志有兼容性重叠，V9/V10 用于模型和测试数据修复。这些是合理的追加式修复，但新开发者不能只看最新 Entity，必须阅读迁移顺序理解已部署环境。

CHAPTER

十一、开发者快速理解总结

本章要点

新开发者先掌握请求上下文、数据范围和两套告警链，再进入具体业务代码。任何扩展都应先回答事实源、权限、事务和数据生命周期四个问题。

1. 建议的代码阅读顺序

1. `frontend/src/utils/request.js`、`router/index.js`、organization/scope Store：先理解请求如何携带登录和组织上下文。
2. `SecurityConfig`、JWT 过滤器、`OrganizationContextFilter`：理解请求进入业务前发生什么。
3. `AuthServiceImpl`、组织申请/成员 Service：理解全局账号与组织关系。
4. `DataScopeServiceImpl` 和 `DataScopeContext`：理解数据范围，这是多数“无权访问”问题的根源。
5. `DeviceServiceImpl`、CloudRoom Service：理解资源归属。
6. Point/Template/Binding/Metric Sample/Latest：理解产品化遥测主链。
7. `DeviceDataServiceImpl` 与 `AlertRuleEngine`：理解仍在运行的旧链。
8. `AlarmEventServiceImpl`、接收人解析、站内通知、WebSocket：理解目标告警闭环以及当前断点。
9. Simulation Service：理解异步、事务提交后执行和上下文传播。
10. `GlobalExceptionHandler`、`AuditLogAspect`、Cache、定时任务和迁移：理解故障表现和运行维护。

2. 新开发者必须建立的四个认知模型

整体架构

一个部署单元、一个数据库、逻辑多模块。不要把 Redis/WebSocket/异步线程误认为分布式服务。

核心业务流程

账号先全局存在，组织通过申请和成员关系产生；每个组织请求由 header 选择上下文；资源归属决定指标、告警和仿真范围；实时消息只提示变化，REST/数据库仍是事实源。

核心模块

账号与组织是治理基础，资源是业务坐标，测点指标是数据主链，告警通知是处置主链，仿真是验证工具，旧遥测告警是兼容层。

关键技术设计

权限采用两层校验，指标采用历史+最新双表，告警采用当前状态+不可变历史，外部副作用在事务提交后执行，Redis 只保存可过期或可重建状态。

3. Debug 快速决策路径

```
接口 401？
  → 检查 JWT、黑名单、用户状态。
接口 403/“无权访问”？
  → 检查 X-Organization-Id、ACTIVE membership、membership_role、DataScopeContext、资源 organization/cloudRoom。
数据写入成功但页面不更新？
  → 先确认写的是 device_data 还是 metric_sample，再检查对应查询和 metric_latest。
有旧 alerts 但没有新告警？
  → 当前缺少 alerts/rule 到 alarm_event 的生产桥接。
告警状态已变但没收到通知？
  → 查 in_app_notification、接收人解析、STOMP Principal 和 /user/queue/summary。
仿真启动 500？
  → 查 write-enabled/profile、安全范围、场景快照、异步 Run 状态和 MetricSample 校验。
中文乱码？
  → 同时检查数据库/表/连接 utf8mb4，并判断历史数据是否已在错误编码下损坏。
```

4. 功能扩展前的边界检查

新增功能前至少回答：

- 它属于旧兼容链还是产品化主链？
- 权威事实存在哪张表，是否还有另一套重复事实？
- 平台角色、组织角色和机房范围分别如何限制？
- HTTP、后台任务和异步调用是否都能复用同一 Service 规则？
- 事务提交前后有哪些外部副作用？
- WebSocket 丢消息后能否从数据库恢复？
- 数据是否有清理、归档或软删除策略？
- 并发请求下状态更新是锁、CAS、乐观锁还是幂等 upsert？

5. 当前最合理的下一步

从架构完整性看，优先级不是立即拆微服务，而是：先打通 `Metric/Rule → AlarmEvent → Notification/WebSocket`，统一 `ORG_ALL` 和接收人权限语义，实现指标保留策略，再逐步把旧 `device_data/alerts` 收敛为兼容适配层。完成这些后，系统才具有一致的产品化监控闭环。

CHAPTER

附录 A：关键代码目录索引

关注点	主要位置
应用入口与配置	`src/main/java/com/example/platform/`、`src/main/resources/application*.yml`
安全与上下文	`security/`、`config/SecurityConfig.java`、`security/CurrentOrganizationContext.java`
Controller	`controller/`
Service 接口与实现	`service/`、`service/impl/`
数据模型	`entity/`、`dto/`
数据访问	`mapper/`
异常与统一返回	`common/exception/`、`common/Result.java`、`common/ResultCode.java`
审计与过滤器	`common/aspect/`、`common/filter/`
定时任务	`task/`
数据库迁移	`src/main/resources/db/migration/`
前端 API	`frontend/src/api/`
前端状态	`frontend/src/stores/`
前端路由	`frontend/src/router/index.js`
前端请求与实时连接	`frontend/src/utils/request.js`、`frontend/src/api/realtime.js`
后端测试	`src/test/java/`

CHAPTER

附录 B：数据库迁移认知索引

迁移	主要作用
V1	基础用户、设备、旧遥测、旧告警等核心 schema
V2	组织、机房、数据范围与资源组织化
V3	测点定义、模板、绑定、指标历史/最新和保留策略
V4	新告警事件、状态转换、处理记录和站内通知模型
V5	审计日志结构化字段
V6	通知日志兼容补充
V7	邮箱身份、注册设置和首次初始化支持
V8	组织成员关系、创建/加入申请和角色约束基础
V9	仿真模型修复与补全
V10	历史设备测试数据乱码/污染修复

阅读迁移时应按顺序理解“系统如何演进”，不能用修改 V1-V6 的方式重写历史。新环境从头执行全部迁移，已有环境只执行尚未应用的新版本。